SISMID 2026

Human–AI Teaming Approach to Infectious Disease Modeling

INSTRUCTORS

Lior Rennert, PhD
Nathan McNeese, PhD
Amanda Bleichrodt, PhD

Introductions

Dr. Lior Rennert, PhD

Instructor

Dr. Nathan McNeese, PhD

Instructor

Dr. Amanda Bleichrodt, PhD

Instructor

Course Outline

A three-day, hands-on path from forecasting foundations to AI-refined models.

DAY 1
Foundations & Setup

- Overview of forecasting
- Intro to AI & human–AI teaming
- GitHub & VS Code setup
- Data cleaning (Lab 1) + GitHub Commits

DAY 2
Prompting & Forecasting

- AI prompting strategies
- Working with rules.md files
- Data visualization (Lab 2)
- Building the forecasting pipeline
- Forecasting (Lab 3)

DAY 3
Evaluation & Refinement

- Overview of model evaluation
- Model evaluation (Lab 4)
- AI/HAT evaluation
- Working with AI for refinement
- Model refinement (Lab 5)
- Questions & free coding

Goals of the Course

What it is, and isn't

- Get comfortable with AI-supported modeling and Human–AI Teaming (HAT)
- Infectious disease forecasting is our case study, not the end goal
- Focus is critical thinking and best practices, not definitive implementation guidelines

Expectations with AI

- AI isn’t perfect, and there’s no single “right” approach
- A human expert is essential for validation, *how do we know the output is correct?*
- AI will keep improving; the aim is experience you can adapt to your own work
- Practical note: managing tokens (and what to do if you run out)

How we'll work

- Most of our time will be spent in hands-on labs, where we will show you multiple ways of doing things
- Mix of class-led walkthroughs, guided solo exercises, and free-flow coding
- Don’t be discouraged if something breaks or is unclear, you can always ask

HAT-based approach for this course: Instructors will create structure for AI-based implementation, class will help refine approach through collaboration with each other, instructors and AI

Life Secret: Instructors don’t know everything

Infectious Disease Forecasting

July 22, 2026

Infectious Disease Forecasting: Outline

Concepts & Methods

- Background
- Model components
- Model types: ARIMA, GAM, XGBoost, ensemble
- Incorporating predictors
- Training & testing periods
- Model evaluation
- Model refinement

Use Case: Influenza Forecasting

CDC NHSN inpatient admissions

- Data visualization
- Model building
- Training & testing
- Evaluation
- Refinement
- FluSight submission

Background

- **Infectious disease forecasting** is a data-driven field that uses mathematical, statistical, and AI models to predict the trajectory, timing, and severity of disease outbreaks.
  - Like weather forecasting, it gives decision-makers a probabilistic look ahead.
- **Informs decisions:** situational awareness, planning, real-time response and resource allocation, and policy.
- **Widely used in practice:** efforts like CDC FluSight, COVID-19 and RSV Forecast Hubs combine many models to guide national response.
  - Also used by hospitals and healthcare entities for surge planning and response
- **Inherently challenging:** data are noisy, delayed, and revised; human behavior shifts; novel pathogens offer little historical signal; and existing pathogens often lack real-time leading indicators

Components of Infectious Disease Models

In this course, we focus on statistical and machine learning models. Their typical components include:

Forecast target

What we predict: cases, hospitalizations, deaths, peak timing, or epidemic size.

Input data

Surveillance signals: E.g., cases, test positivity, digital traces, admissions, waste-water, vaccination, or mobility.

Model form

The math linking inputs to the target: e.g., regression model

Observation process

Adjusts for imperfect data: underreporting, delays, noise, and revisions.

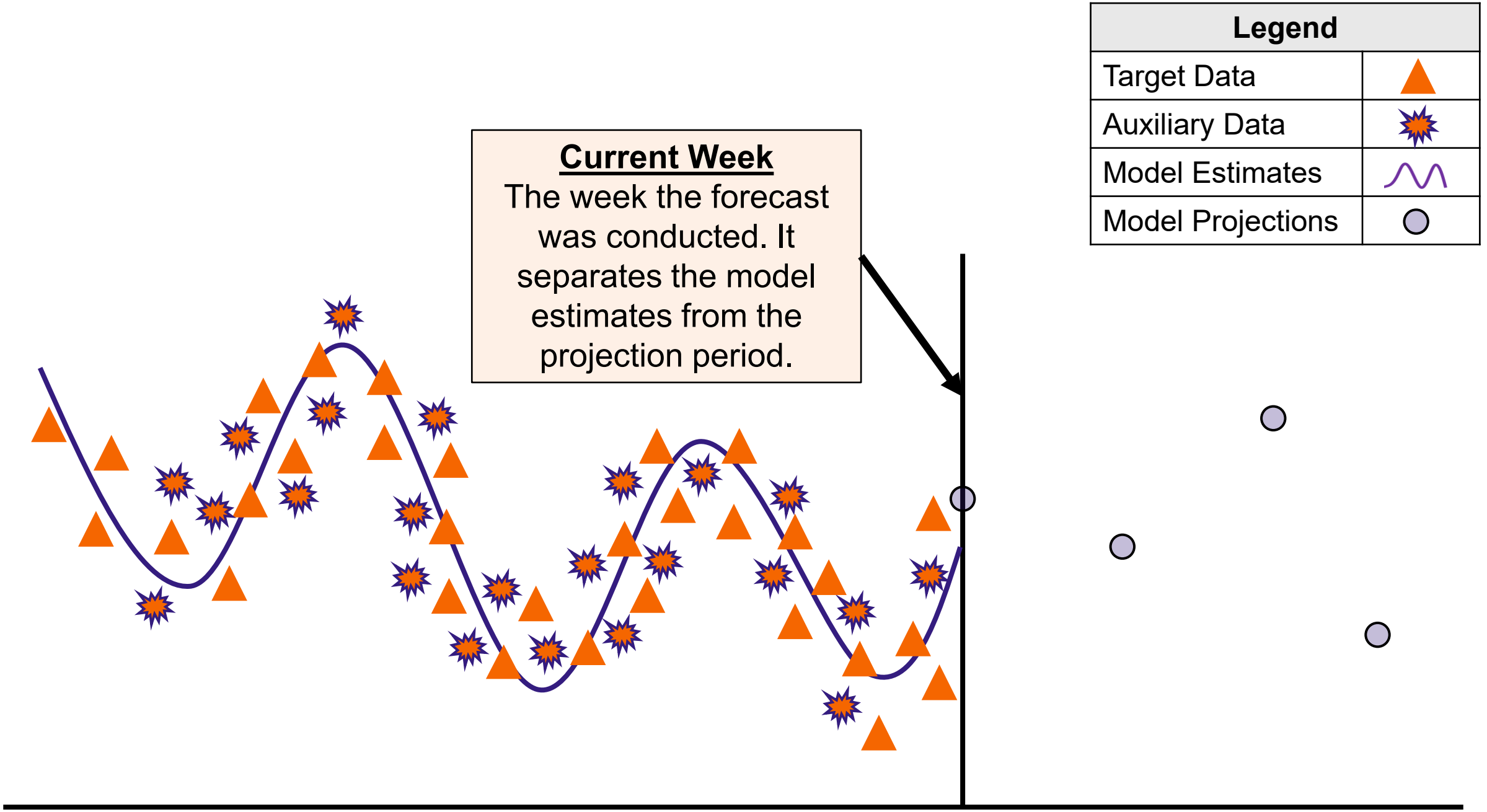
Uncertainty estimates

Prediction intervals or probabilistic forecasts: a range of plausible futures.

Forecast horizon

How far ahead we predict, e.g., 1 to 4 weeks; accuracy drops further out.

Building a Forecast

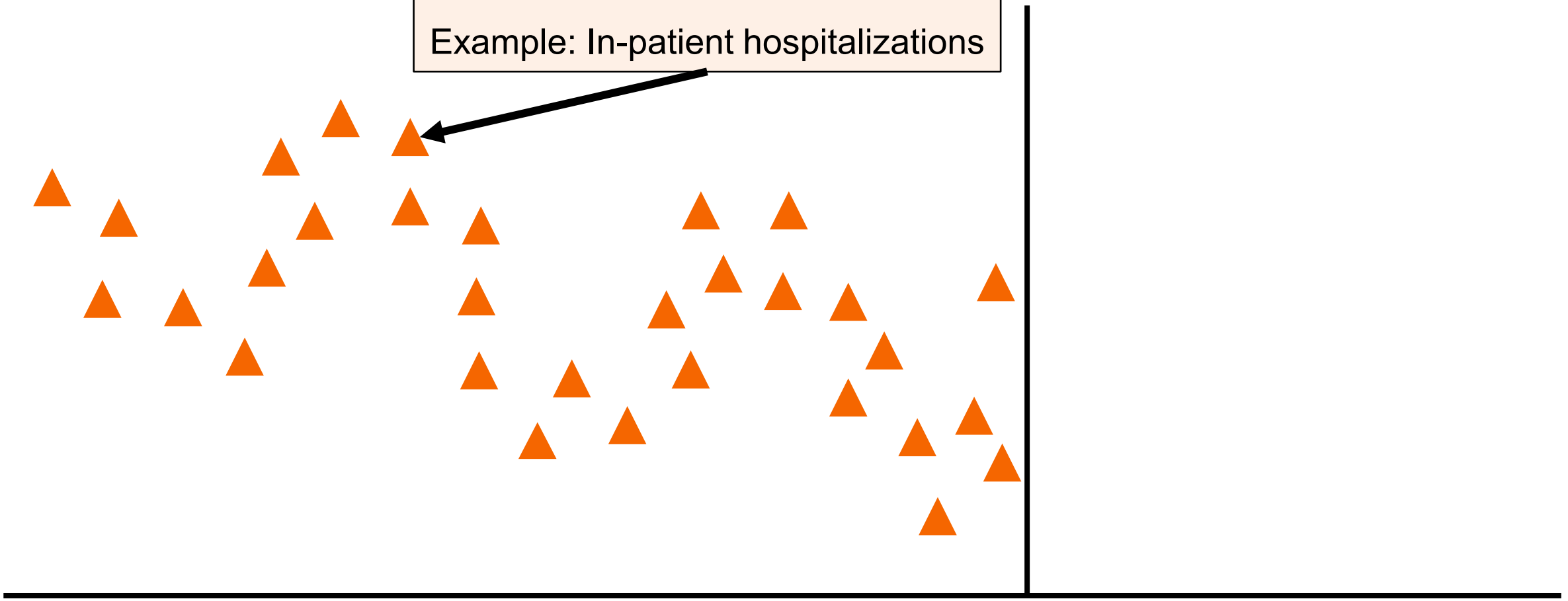

Legend
Target Data
Auxiliary Data
Model Estimates
Model Projections
Current Week
The week the forecast was conducted. It separates the model estimates from the projection period.

Target Data

The historical data the model learns from and uses to estimate its parameters; it represents the **outcome** being forecasted by the model.

Example: In-patient hospitalizations

Legend	
Target Data	▲

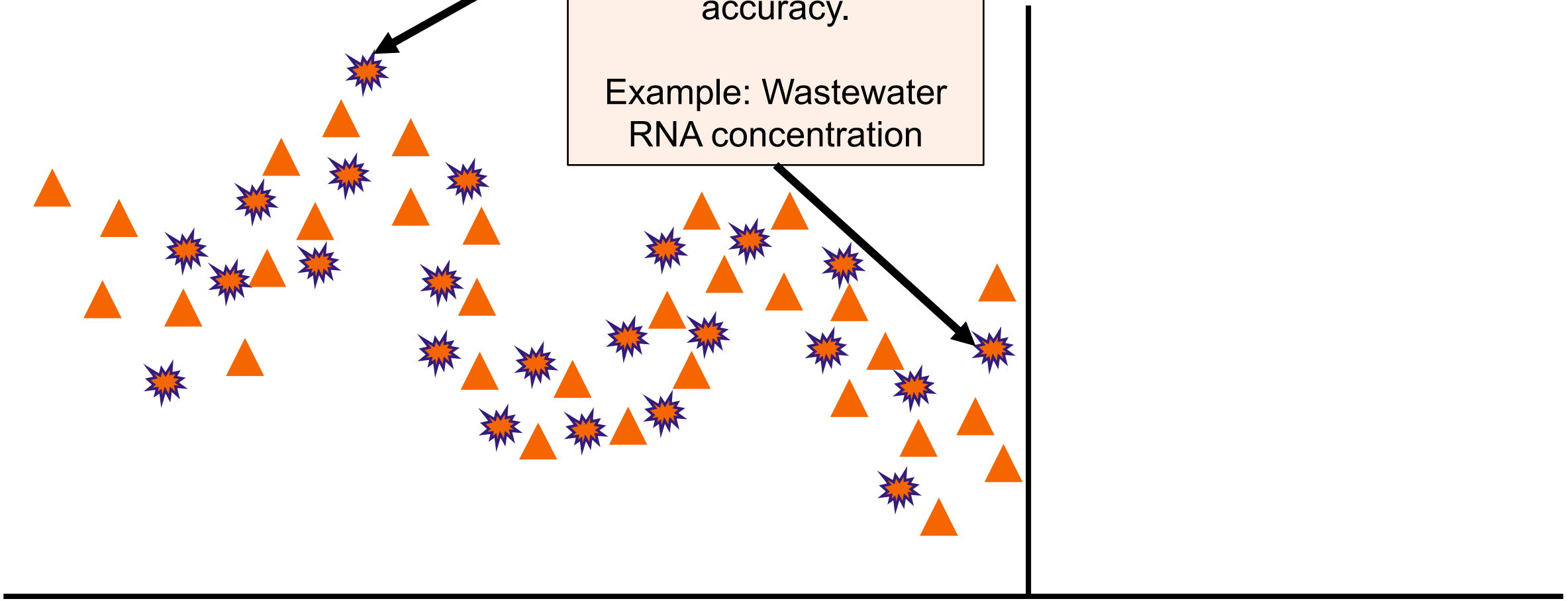


Auxiliary Data

Additional datasets incorporated as predictors or supporting variables in the model to enhance its predictive accuracy.

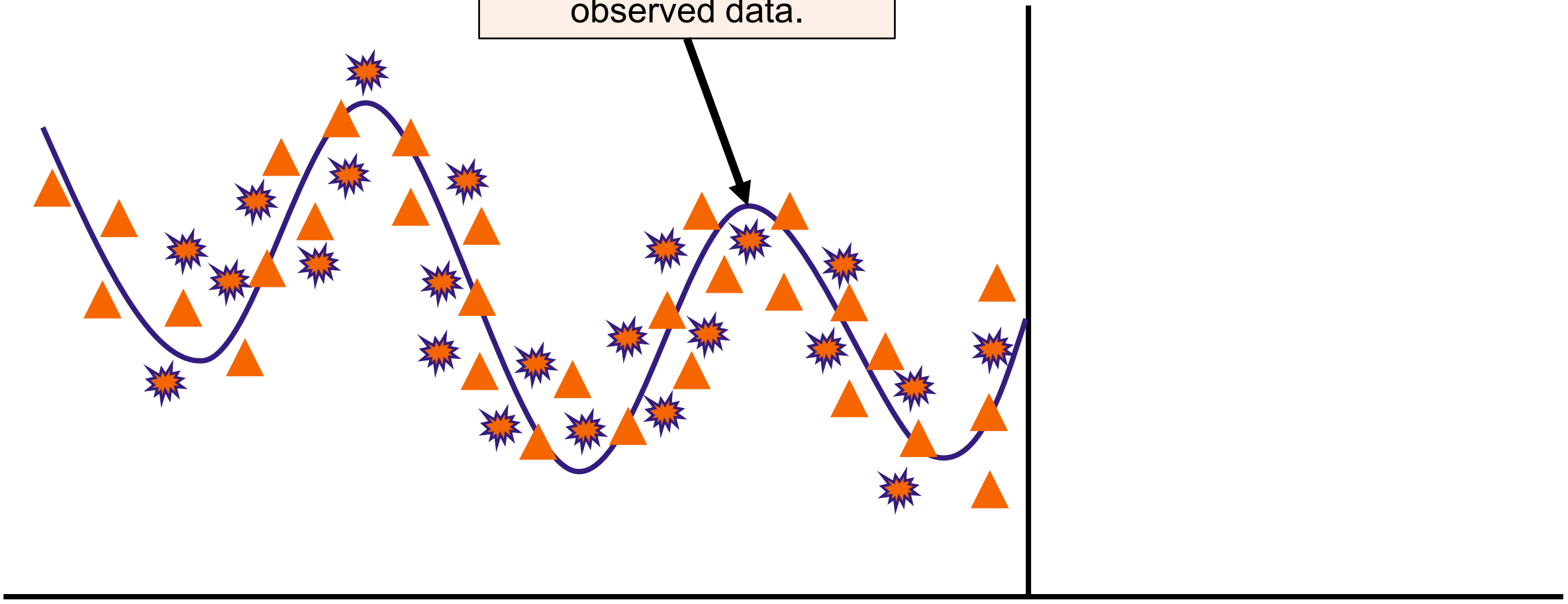
Example: Wastewater RNA concentration

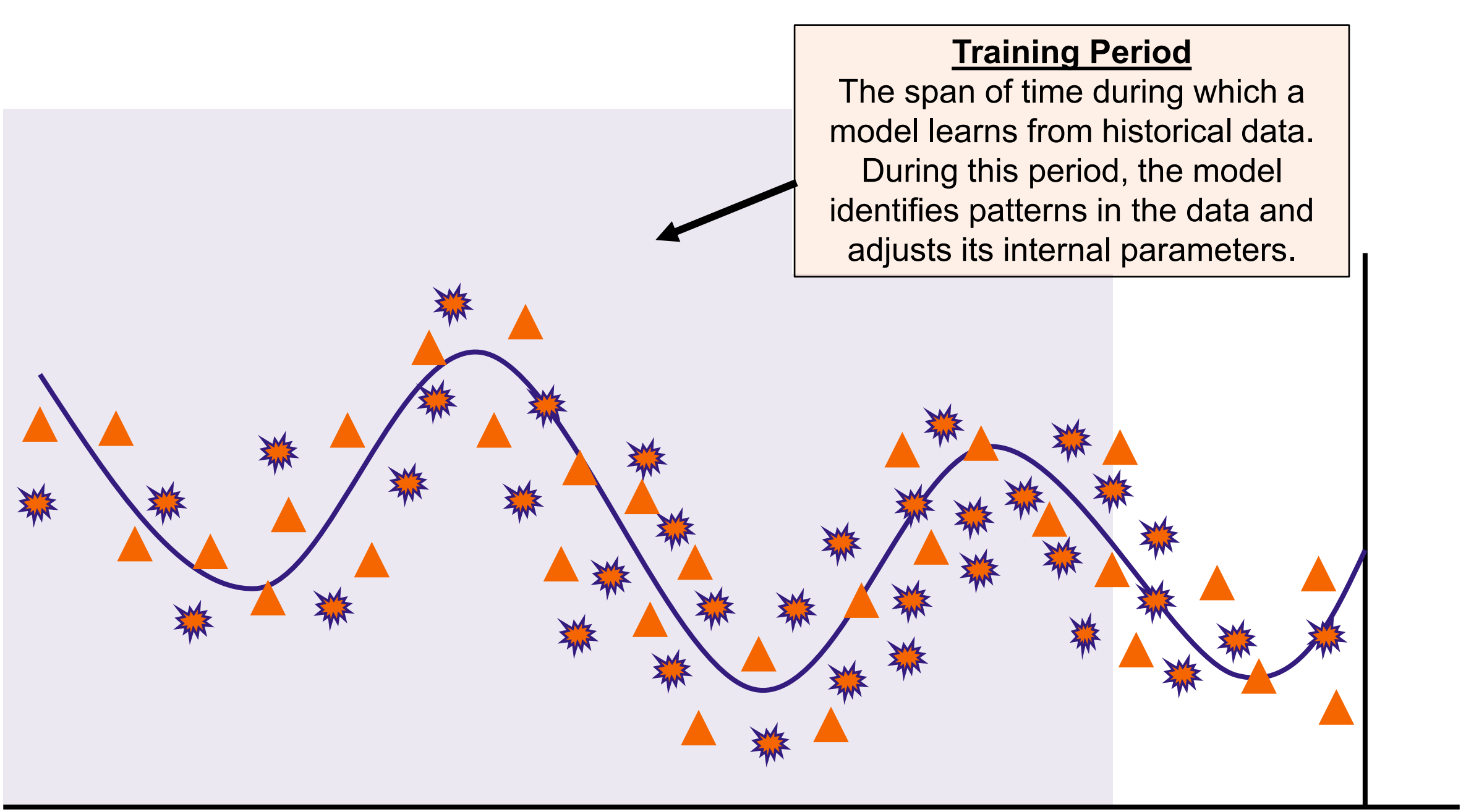
Legend	
Target Data	▲
Auxiliary Data	✸



Legend	
Target Data	
Auxiliary Data	
Model Estimates	

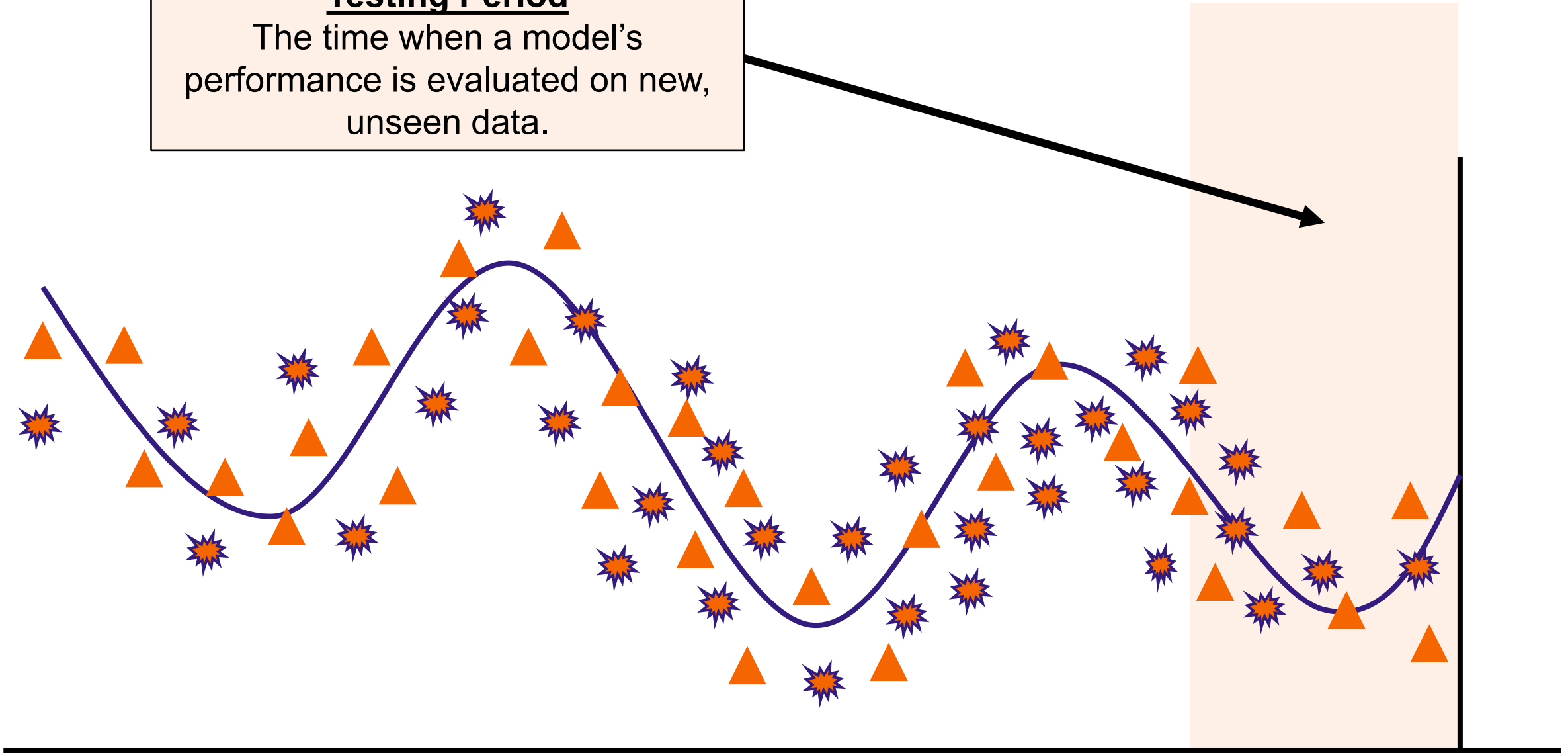
Model Estimates
Historical model outputs for periods with observed data.




Training Period
The span of time during which a model learns from historical data. During this period, the model identifies patterns in the data and adjusts its internal parameters.

Testing Period

The time when a model's performance is evaluated on new, unseen data.



Evaluation Example: Error

Typically evaluated in the testing period

Example: $Percentage\ Error = \frac{|P-O|}{O}$

O = Observed counts of the outcome of interest across the evaluation period
P = Predicted counts of the outcome of interest across the evaluation period

Example: Our model forecasted 1000 cases of influenza across four weeks. However, 1500 cases were observed during those same four weeks.

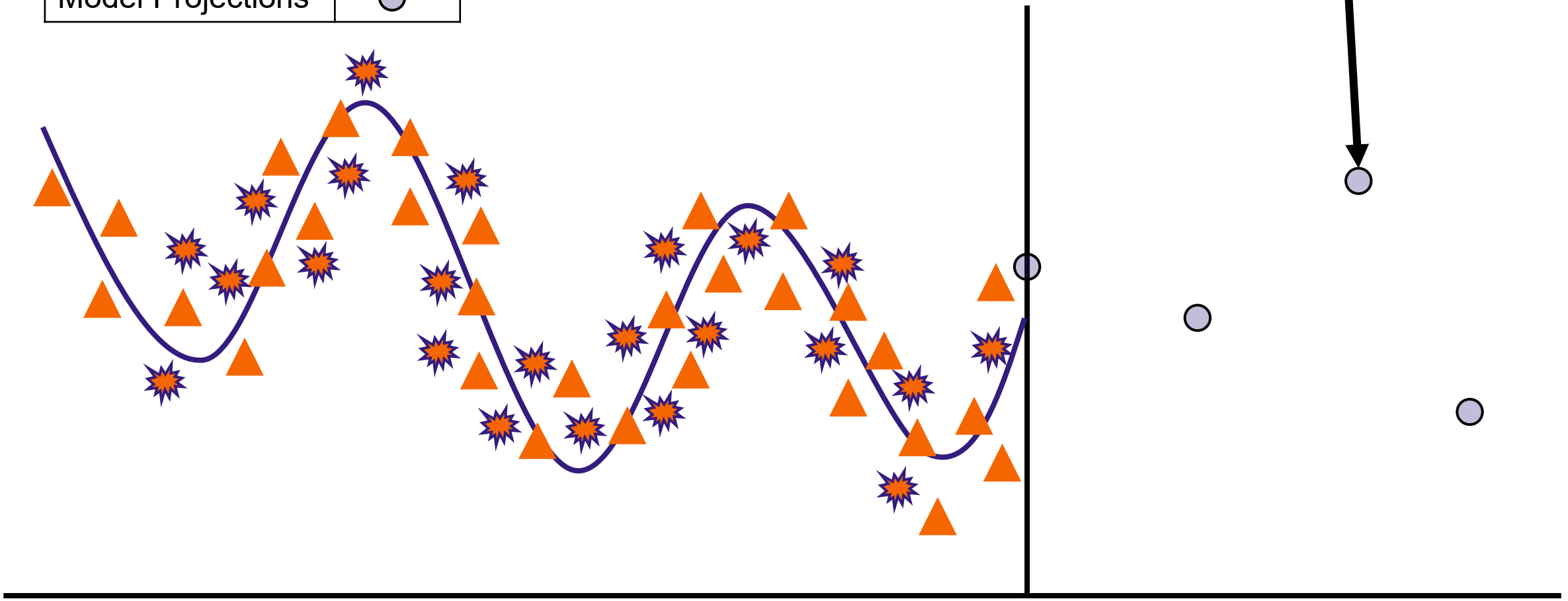
$$O = 1500\ cases$$
$$P = 1000\ cases$$

$$Percentage\ Error = \frac{|1000 - 1500|}{1500} = \frac{500}{1500} = 33\%$$

Question for class: Can we identify a limitation of this metric?

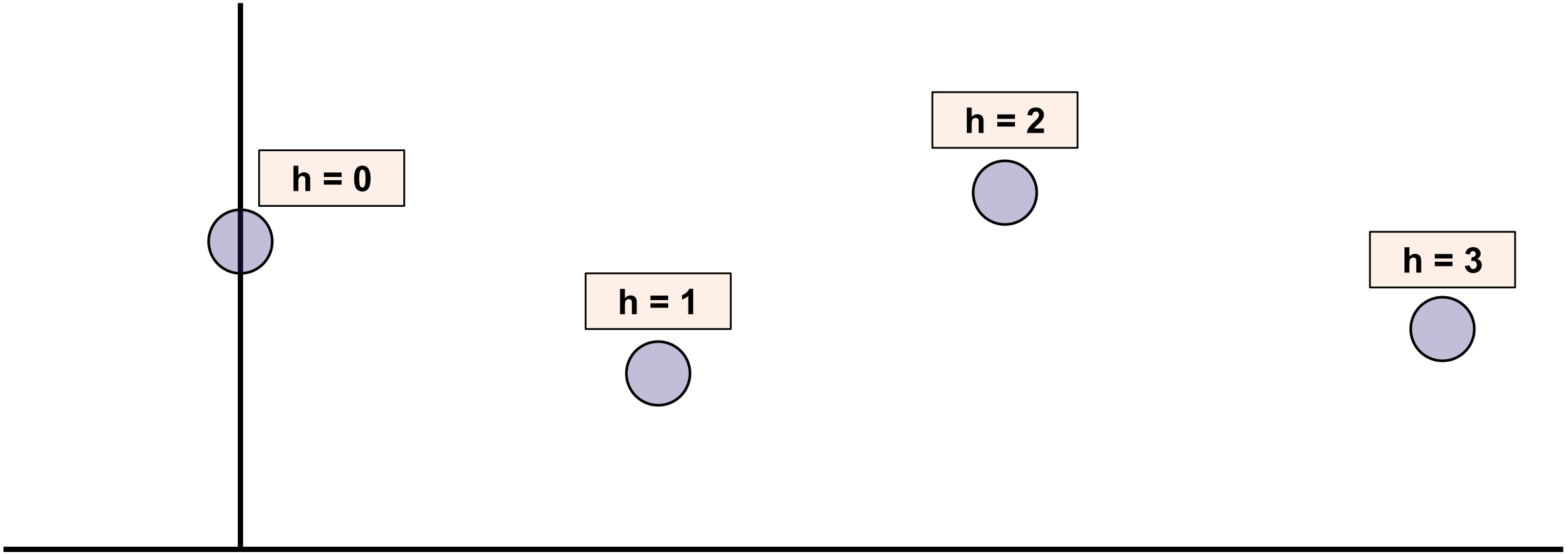
Legend	
Target Data	
Auxiliary Data	
Model Estimates	
Model Projections	

Model Projections
Model-generated values for future periods without observed data.



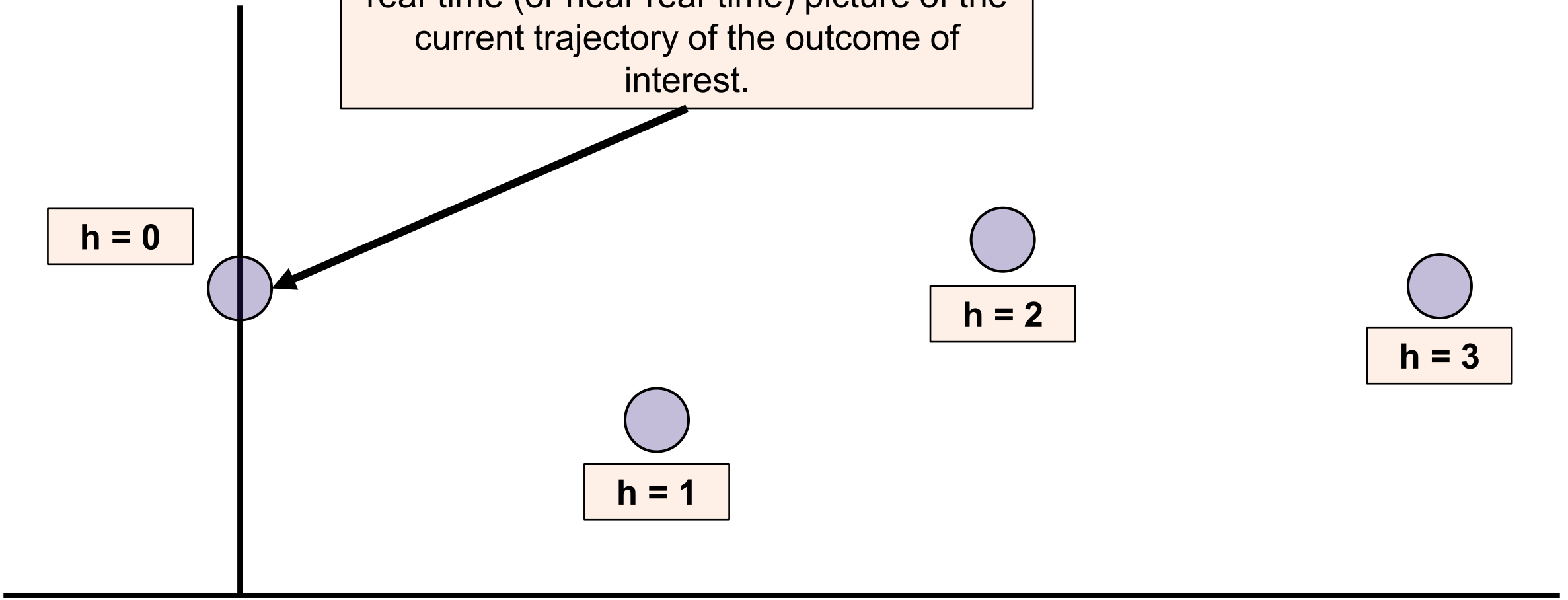
Projections Terminology: Horizons

Forecasting Horizon (h): The number of future time points (e.g., weeks, days) from the "current" reference point for which a forecast provides predictions. It defines how far into the future the model is attempting to project the outcome of interest.



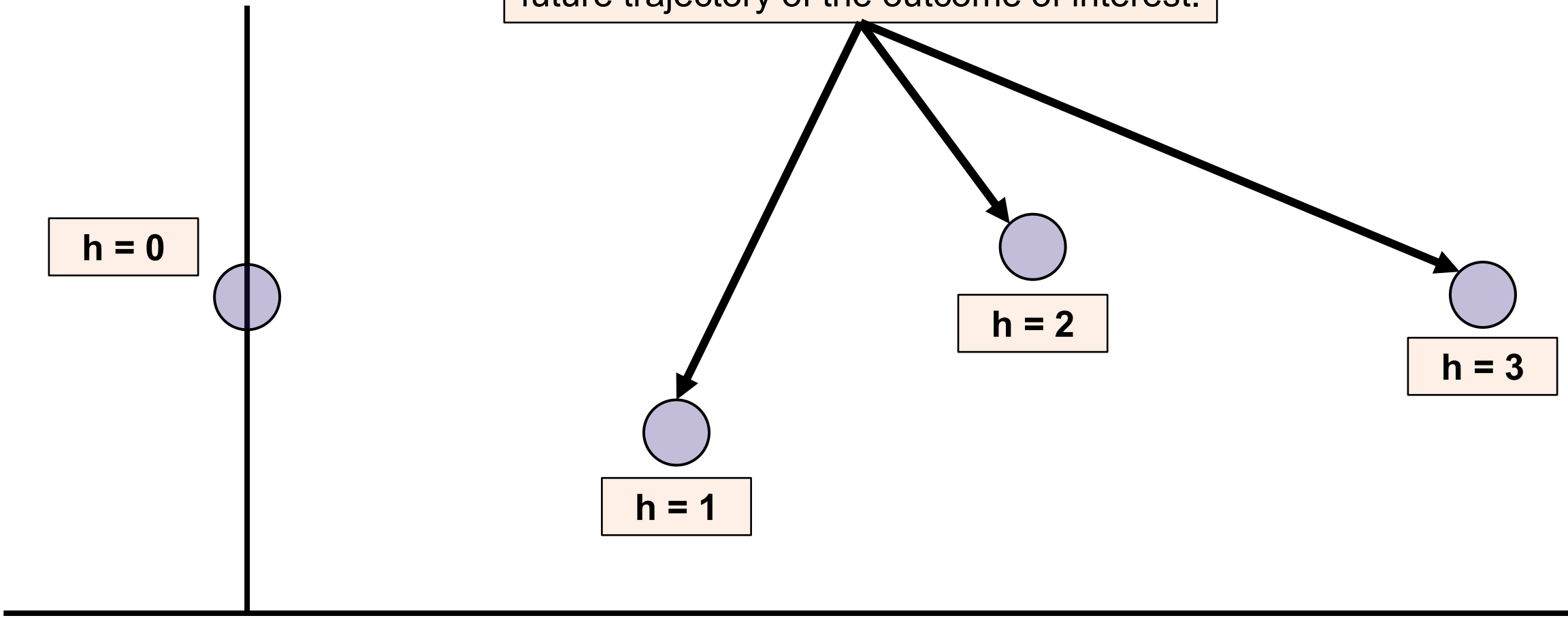
Projections Terminology

Nowcast
A “right-now” prediction that provides a real-time (or near real-time) picture of the current trajectory of the outcome of interest.



Projections Terminology

Forecast
A prediction that provides a view of the future trajectory of the outcome of interest.



Infectious Disease Models

Overview

Infectious disease models span a spectrum, from statistical methods to machine learning and other combined-model approaches.

ARIMA - Primary	GAM	XGBoost	Ensemble
Models a series from its own past — autocorrelation, trend (via differencing), and seasonality.	Flexible regression using smooth spline terms to capture nonlinear trends and seasonality.	Gradient-boosted decision trees that learn from engineered features and lagged predictors.	Combines several models' forecasts (mean, median, or weighted) — usually the most robust.

Example Model Overview(s)

ARIMA — Statistical

Models a series from its own past — autocorrelation, trend (via differencing), and seasonality.

AutoRegressive Model

- At its core, an AR model is just **linear regression** that uses the series' own recent **values** to predict what comes next.

$$Y_t = \alpha + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \varepsilon_t$$

Y_t	Value *Y* (our outcome) at current time t
α	Intercept Term; Estimated by model
p	How many past *values* we look back at
ϕ_p	Regression coefficient: weight given to the value from p periods back; estimated by the model
Y_{t-p}	Value of *Y* (our outcome) p time steps back
ε_t	The random error at time t

Integrated Model

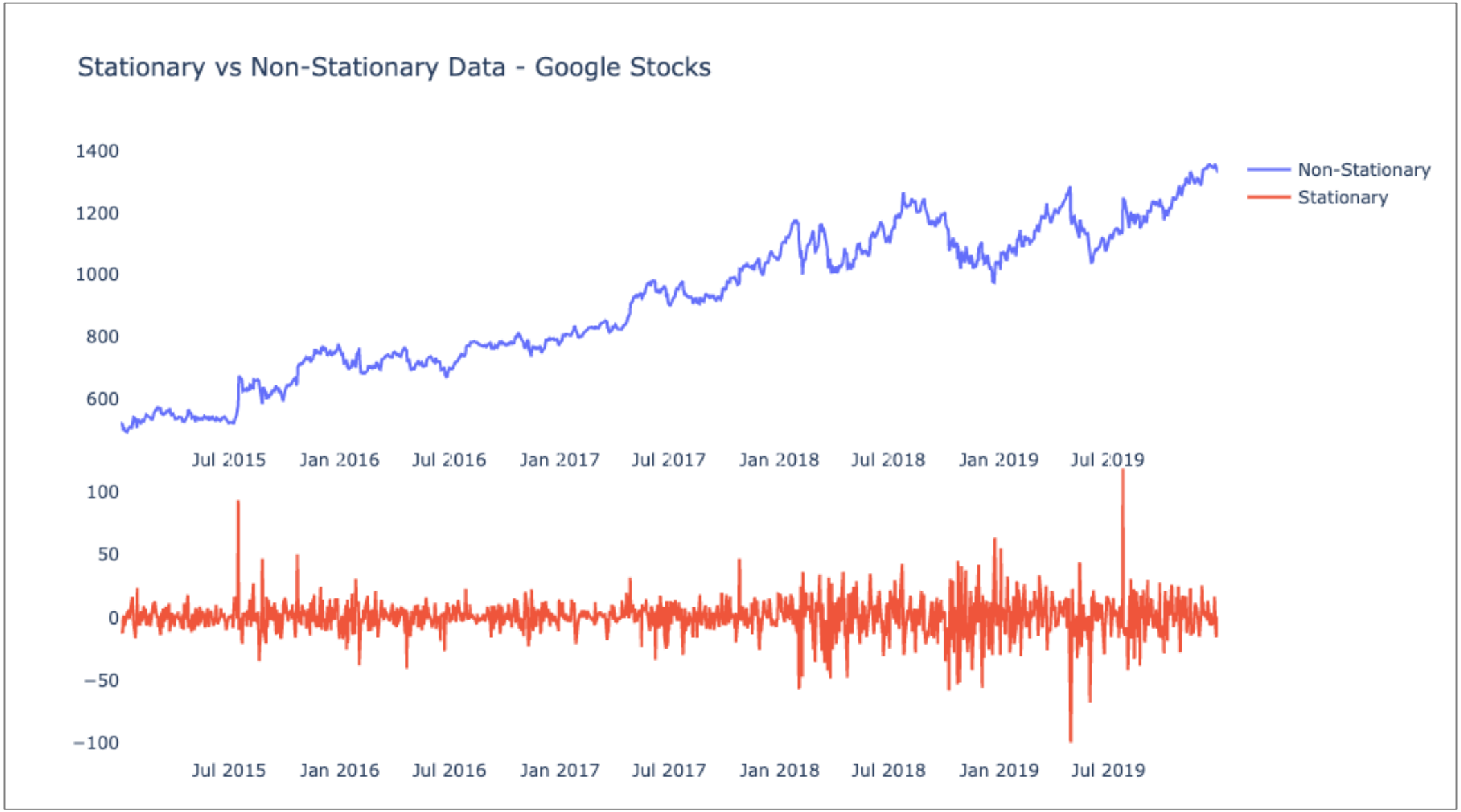
- ARIMA requires the time series to be made **stationary**

What is a stationary time series?

A time series is **stationary** if its statistical behavior doesn't change over time (e.g., constant mean, variance)

- Integration means **differencing** the data: subtracting each value (Y_t) from the one before it ($Y_t - Y_{t-1}$) to strip out trend and make the series stationary.
- Parameter d indicates how many times you **difference** the data

Example: Stationary Time series



Wise, *Time Series…*, 2020

Moving Average Models

- In an MA model, the current value is a **linear** combination of recent (current and past) random errors ("white noise").

$$Y_t = \alpha + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

Y_t	Value *Y* (our outcome) at current time t
α	Intercept Term; Estimated by model
q	How many past *errors* we look back at.
θ_q	Regression coefficient: weight given to the errors from q periods back; estimated by the model
ε_t	The random error at time t
ε_{t-q}	The random error q time steps back

ARIMA Models

- **ARIMA:** Autoregressive Integrated Moving Average
- All three components combine to create an **ARIMA** model (*AR* + *I* + *MA*).

$$Y_t{}' = \alpha + \phi_1 Y'_{t-1} + \cdots + \phi_p Y'_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

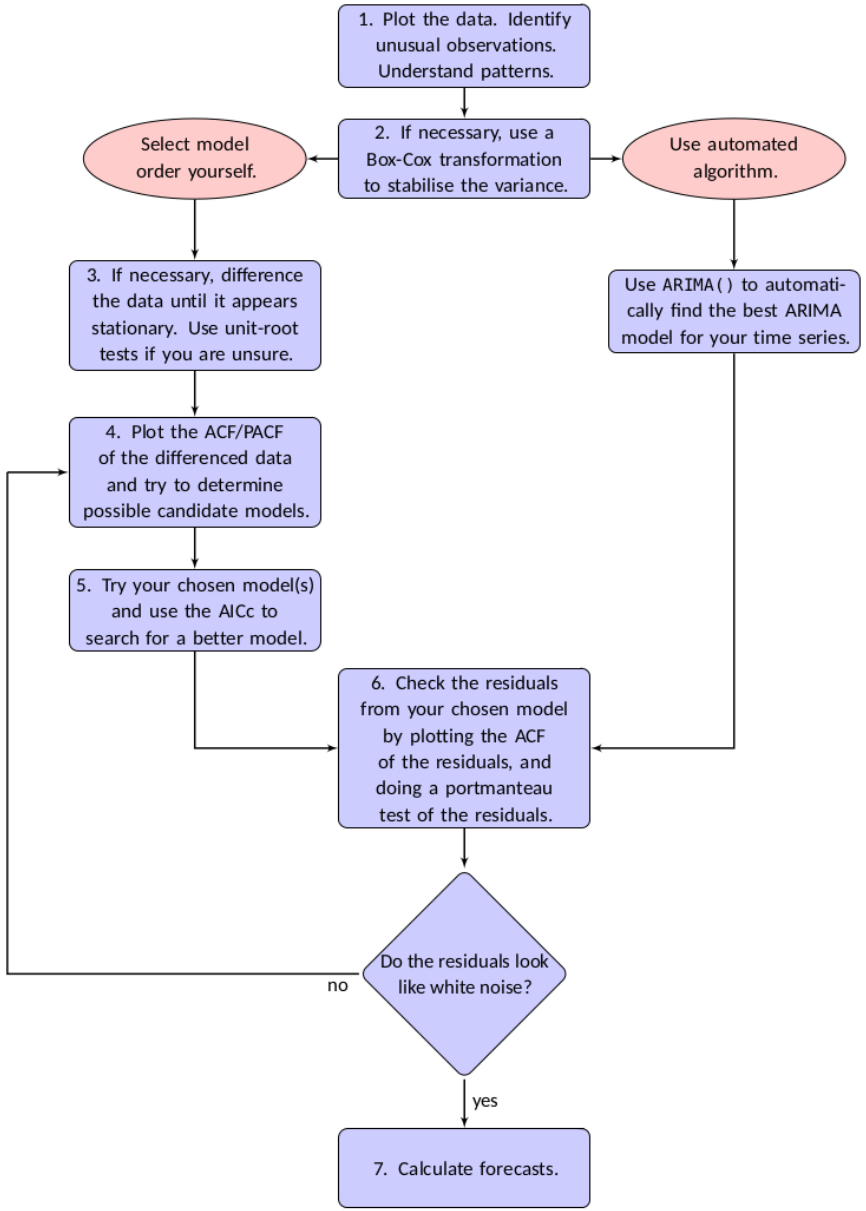
New Notation	
Y'_t	The value of the *differenced* series at current time t
Y'_{t-p}	Value of *the differenced* series p time steps back

In words: Predicted differenced value at time t = constant + linear combination of recent (differenced) values + linear combination of recent forecast errors

Parameters {*p, d, q*}

Parameter	Model	Description
p	AR	How many **past values** we look back at.
d	I	How many **differences** are needed to make the time series **stationary.**
q	MA	How many **past errors** we look back at.

We will be using the `auto.arima()` function from the `forecast` package. Therefore, the parameter values will automatically be selected via the **Hyndman-Khandakar Algorithm**.



Hyndman-Khandakar Algorithm

1. The number of differences $0 \leq d \leq 2$ is determined using repeated KPSS tests (checks stationarity).

2. *p* and *q* are selected by looking for the combination of parameters (stepwise process) which produce the smallest AICc after differencing the data *d* times.

3. Four models are fit initially.
    1. The model from (1) with the smallest AICc is considered the “current” model.
    2. The current model is varied, and if a better model arises it becomes the “current” model.
    3. Repeat (3.2) until no lower AICc is found.

Hyndman & Athanasopoulos, *Forecasting…*, 2026

Example Model Overview(s)

ARIMA — Statistical

Models a series from its own past — autocorrelation, trend (via differencing), and seasonality.

GAM — Statistical

Flexible regression using smooth spline terms to capture nonlinear trends and seasonality.

Generalized Additive Models

- Very similar to Generalized Linear Models (GLM) but allows for modeling of more complex data trends (i.e., non-linear data) via “smoothing” functions (splines, we will cover these shortly).

$$g(\mu) = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_p(x_p)$$

μ	The value we expect for the outcome.
$g(\cdot)$	The link function connecting predictors to the mean outcome.
β_0	The intercept, or baseline value the model estimates.
$f_p(\cdot)$	Smooth curves (splines) the model learns for each predictor.
x_p	A predictor variable, an input used to explain the outcome.

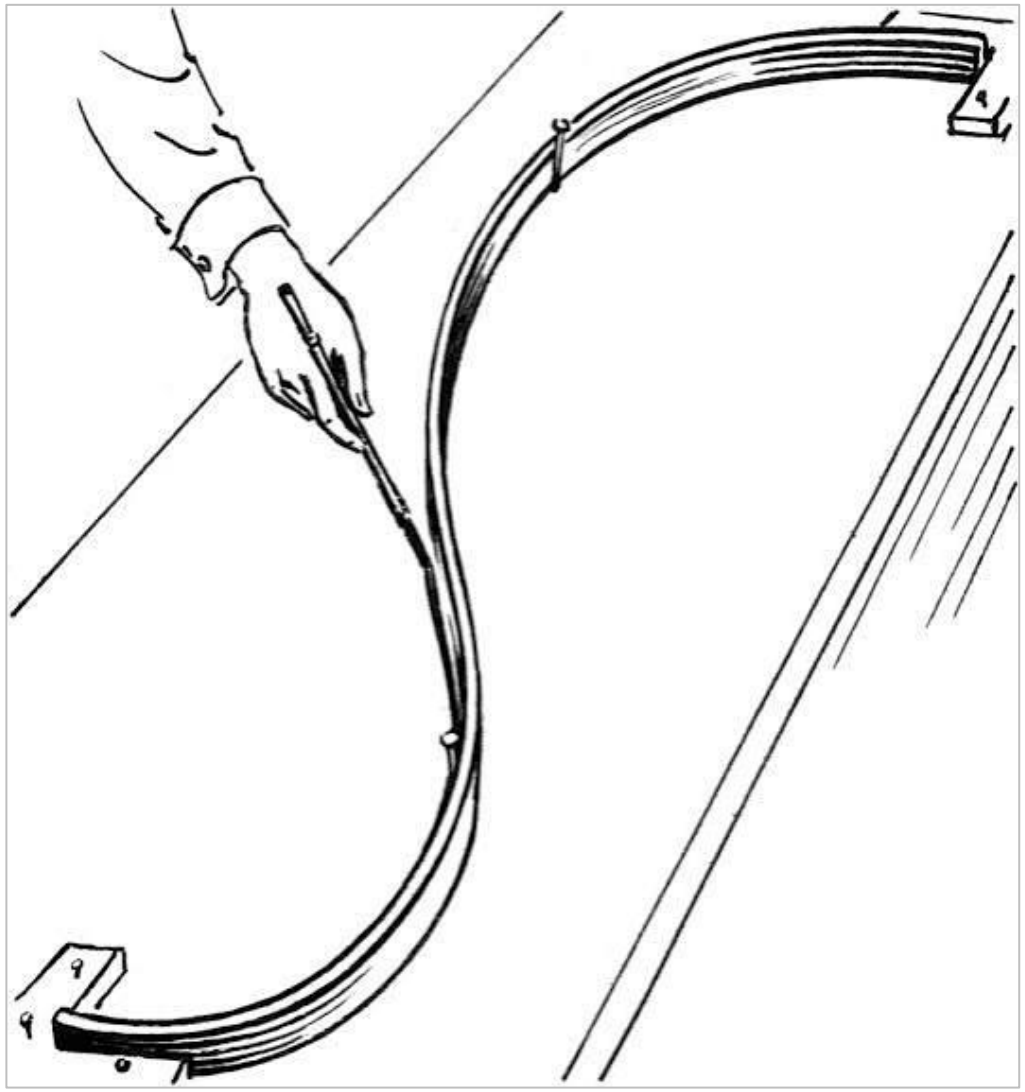
What are splines?

- Smooth, flexible curves made by joining simple polynomial pieces (usually cubics) at points called knots, so it can bend to fit data without kinks.

$$f(x) = \sum_{k=1}^{K} \beta_k b_k(x)$$

$f(x)$	The overall smooth curve we fit to a predictor.
K	The basis dimension: roughly how many basis functions, or how wiggly the curve is allowed to be.
β_k	The coefficients the model estimates. They set how much each basis function contributes (i.e., how much each piece bends).
b_k	The fixed basis functions (e.g., B-splines or thin-plate). They encode the pieces and knot placement.

History of Splines



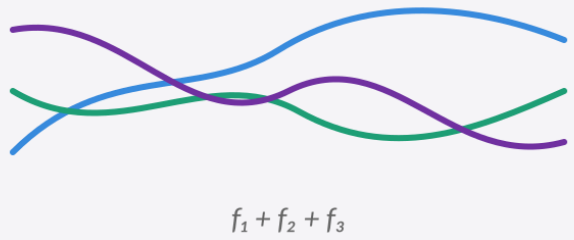
Long before splines were math, they were wood. To draw the sweeping curves of a ship's hull, draftsmen worked in the shipyard's mould loft, using a ***spline,*** *a thin, flexible strip of wood bent to pass through a set of fixed points. The strip was pinned down by heavy weights called* ***"ducks,"*** *and between those points it relaxed into the smoothest curve on its own, because a bending strip naturally settles into the shape that stores the least bending energy. Mathematicians later formalized exactly that idea and kept the name: a spline is the curve that passes through the points while bending as little as possible.*

Papp, *Splines: how do they work*, 2020; WIKI, *Flat spline*, 2025

Putting It All Together

A GAM is only as good as the smooth curves inside it, and a curve flexible enough to be useful is flexible enough to go wrong (**Hint:** wiggliness vs overfitting).

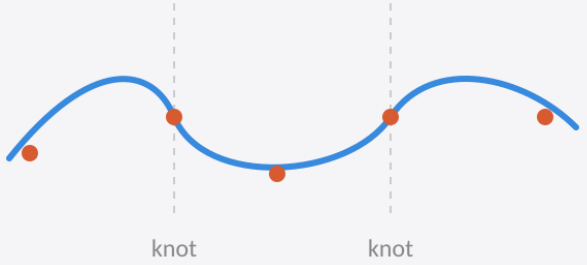
1



A GAM is a sum of smooths

Each predictor gets its own flexible curve, not a straight line.

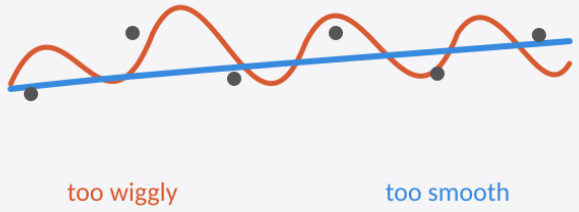
2



Each smooth is a spline

That flexibility comes from joining simple pieces, like the shipyard strip bending through its points.

3

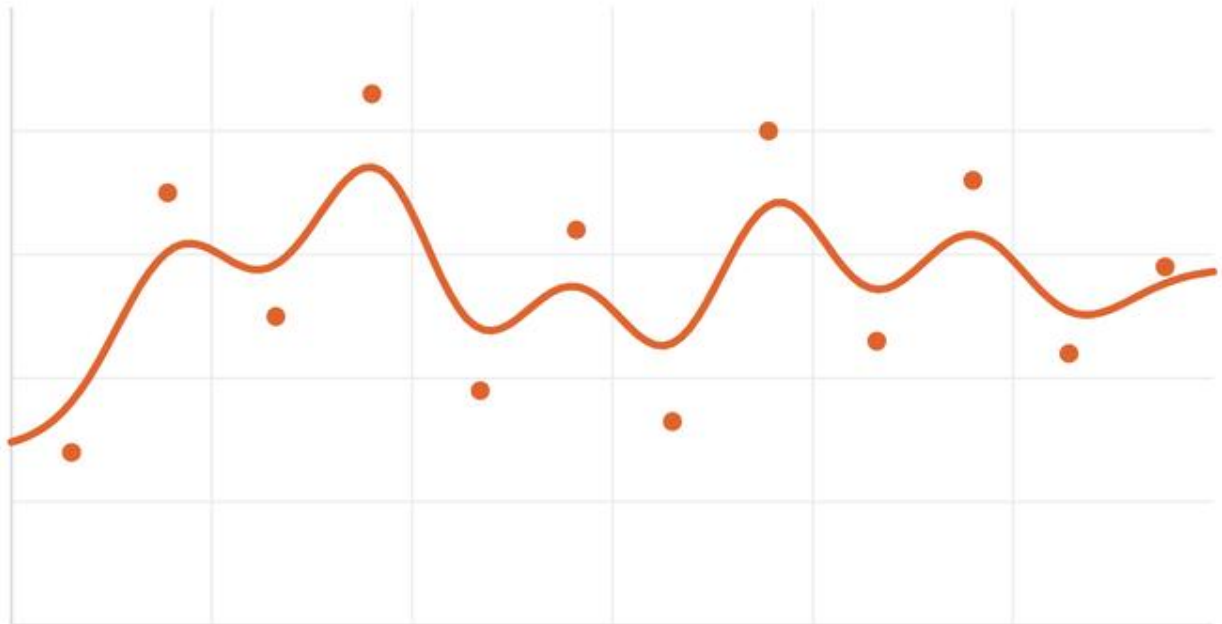


Flexibility cuts both ways

Too wiggly overfits; too smooth misses the signal. A penalty (λ), fit automatically, finds the right balance.

How *λ* and *K* balance fit against wiggliness

K sets how wiggly the curve is *allowed* to be. **λ** dials how much of that flexibility the fit actually uses — from chasing every point to a straight line.



λ smoothing penalty **0.000**

0 · wiggly ∞ · straight

K basis dimension **10**

3 18

current fit

overfit — chasing points

Note: mgcv picks λ for you (REML) — you set K high enough to allow the shape, and the penalty prevents overfitting.

Example Model Overview(s)

ARIMA — Statistical

Models a series from its own past — autocorrelation, trend (via differencing), and seasonality.

GAM — Statistical

Flexible regression using smooth spline terms to capture nonlinear trends and seasonality.

XGBoost — Machine learning

Gradient-boosted decision trees that learn from engineered features and lagged predictors.

XGBoost: Boosting Intuition

XGBoost builds its forecast from many small decision trees, added one at a time:

Start rough, then correct

Begin with a simple guess. Each new tree focuses on what the model still gets wrong (the residuals) and nudges the prediction closer to the truth.

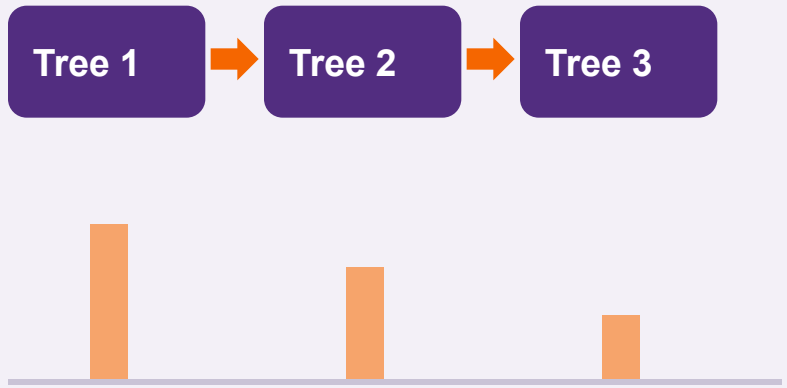
Trees are the building blocks

A decision tree asks simple yes/no questions (e.g. “was last week high?”) and predicts a value for each group it creates.

Two dials keep it honest

Learning rate η sets how big each correction step is (small steps generalize better); **regularization** penalizes overly complex trees to prevent overfitting.

Each tree fixes the last one’s errors



Add trees step by step (Tree 1 + Tree 2 + …). The orange bars show the remaining error shrinking with each tree.

residual what the model still gets wrong · **tree** a simple yes/no prediction rule · **η** learning rate (step size) · **regularization** penalty that keeps trees simple

XGBoost for Forecasting

Trees need a table, so we turn the time series into features:

Turn history into features (X)

Recent lagged values, rolling averages, and calendar flags (week, month, holiday), plus outside signals like labs or wastewater.

Strengths

Learns nonlinear patterns and interactions on its own, handles many predictors, and is strong on tabular data.

Limits to watch

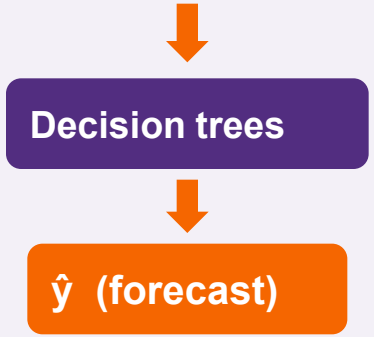
It cannot extrapolate a trend beyond values it has seen, needs enough history, and needs extra methods (e.g. quantile models) for prediction intervals.

In practice

A common, strong machine-learning entry in challenges like FluSight

From a feature table to a forecast

lag	roll	week
y_{-1}	$\bar{y}$	48
y_{-2}	$\bar{y}$	49



X — table of predictors (features) · **lag** — a past value used as input · **rolling average** — a smoothed recent mean · **ŷ** — the forecast the model outputs

Extreme Gradient Boosting

Trees for tabular data: the secret sauce in machine learning

Outline

1. Machine learning algorithms and common use-cases
2. Regression tree and CART
3. Tree ensembles
4. Learning trees: advantages and challenges
5. XGBoost overview
6. XGBoost workflow
7. Core XGBoost terms
8. XGBoost classification intuition
9. XGBoost regression intuition
10. XGBoost advantages and disadvantages

Machine Learning Algorithms and Common Use-cases

Linear Models

for Ads Clickthrough

Factorization Models

for Recommendation

Deep Neural Nets

for Images, Audio etc.

Trees for tabular data

the secret sauce in machine learning

Common Use-cases of XGBoost

- Anomaly detection
- Ads clickthrough
- Fraud detection
- Insurance risk estimation
- ...

Regression Tree and CART

Regression tree

A regression tree is a decision tree that predicts a numeric score. It asks feature questions and sends each record to a final leaf.

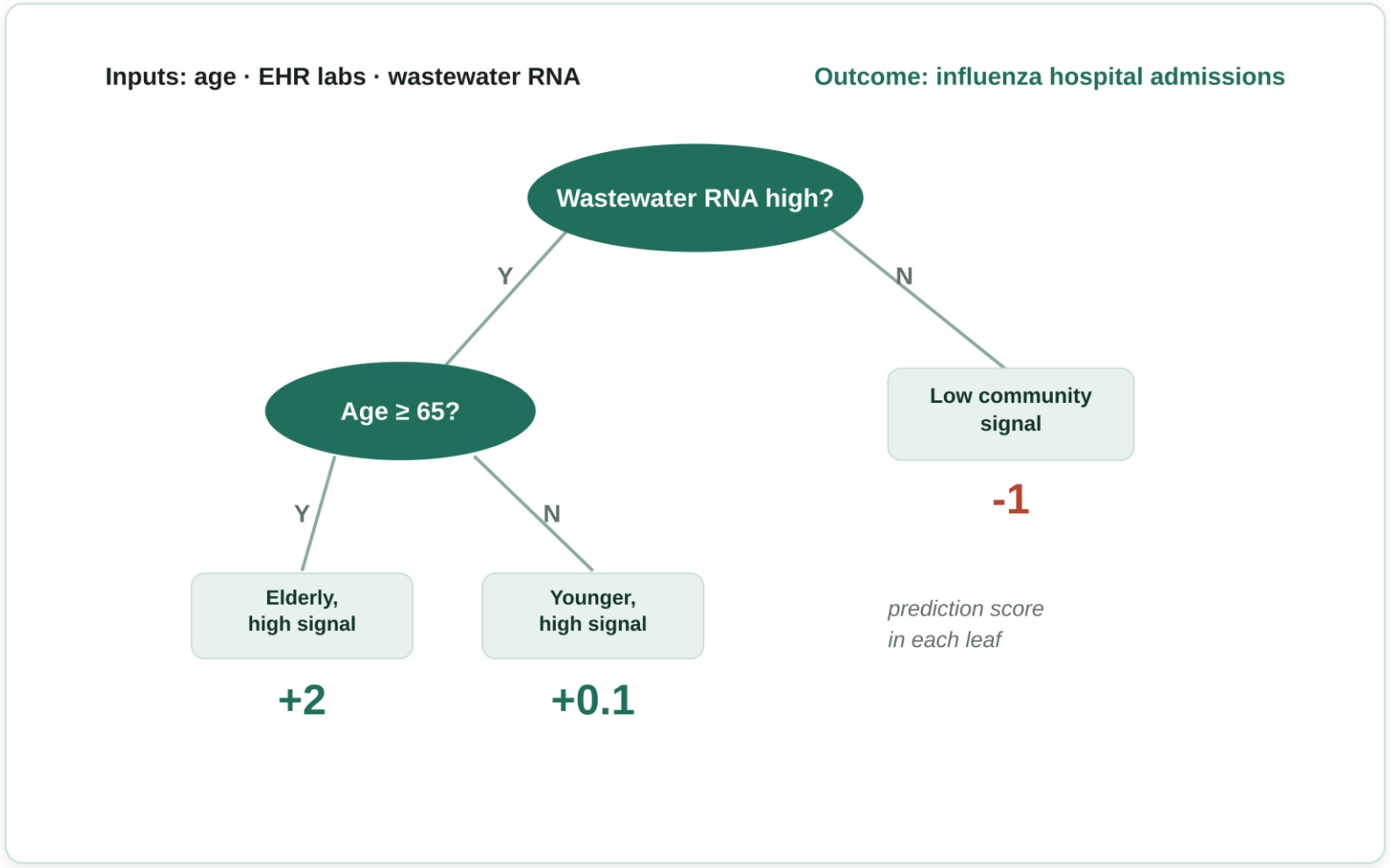
CART

CART means Classification and Regression Tree. It is the tree framework used for both class labels and numeric predictions.

Regression Tree

Regression tree, also known as CART.

Ask a question, follow a branch, land on a score.



How the Regression Tree Reaches a Score

1

Problem

The figure predicts influenza hospital admissions using features such as age, EHR labs, and wastewater RNA concentration.

2

First split

The tree first asks whether the wastewater RNA concentration is high. This separates areas with heavy viral signal from areas with little signal.

3

Second split

Among high-signal cases, the tree asks whether age is 65 or older. This isolates the group most likely to be hospitalized.

4

Leaf score

Each final leaf gives a prediction score: +2 for high-signal elderly cases, +0.1 for high-signal younger cases, and -1 for low-signal cases.

Ensemble Trees

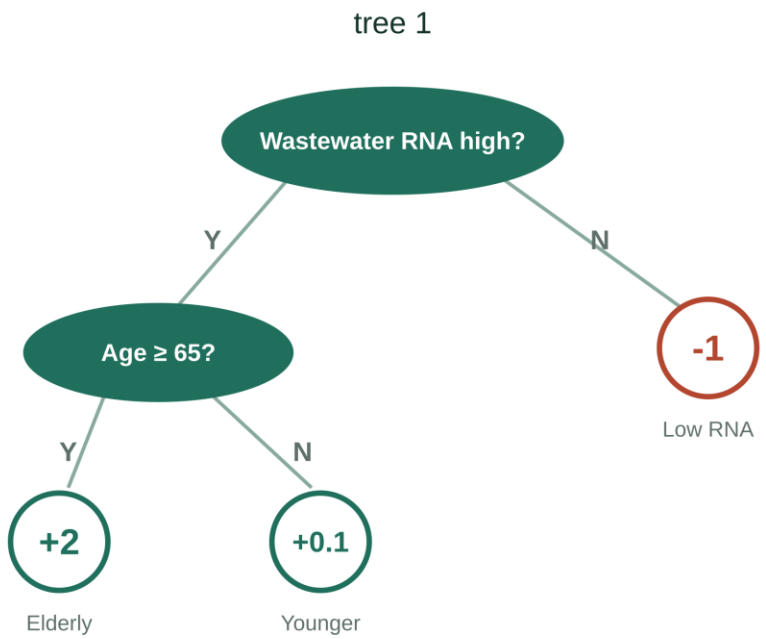
What are ensemble trees?

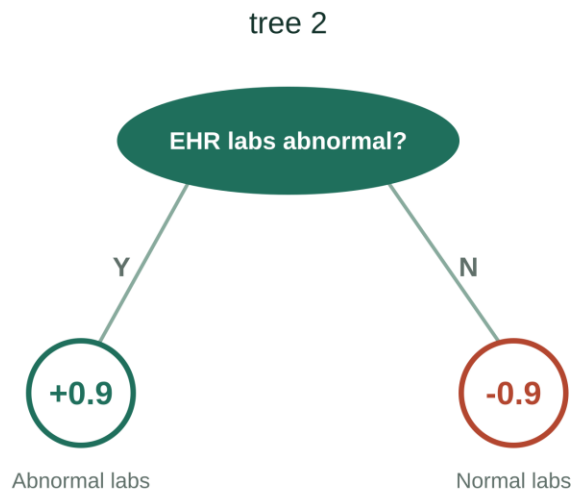
An ensemble combines multiple trees. Each tree gives a score, and the model combines those scores into one final prediction.

Why use an ensemble?

No single tree must be perfect. Different trees can specialize in different parts of the pattern.

When Trees Form a Forest (Tree Ensembles)





f(high-risk case) = 2 + 0.9 = 2.9

f(low-signal case) = -1 - 0.9 = -1.9

How a Forest reaches a score

1

Problem

The model is still predicting influenza hospital admissions for each case.

2

Tree 1

The first tree uses wastewater RNA and age. For example, a high-signal elderly case receives a strong positive score.

3

Tree 2

The second tree uses EHR lab results. Abnormal labs add a positive score, while normal labs subtract score.

4

Final score

The ensemble adds the tree scores. In the figure, the high-risk case gets 2 + 0.9 = 2.9, while the low-signal case gets -1 - 0.9 = -1.9.

Learning Trees: Advantages

Highly accurate for many tabular data problems.

Easy to use because input scaling is usually not required.

Easy to interpret and control compared with many complex models.

Learning Trees: Challenges

Control over-fitting.

Improve training speed.

Scale up to larger datasets.

These three problems are exactly what XGBoost was built to solve.

Traditional models like decision trees and random forests are easy to interpret but may lack accuracy on complex data.

XGBoost (eXtreme Gradient Boosting) is an optimized gradient boosting algorithm that combines multiple weak models into a stronger, high-performance model.

It uses decision trees as base learners, building them sequentially so each tree corrects errors from the previous one and it is known as boosting.

It features parallel processing for faster training on large datasets and allows parameter customization to optimize performance for specific problems.

How XGBoost Works

How XGBoost Weighs Its Trees

1	**Instance**	The top box is one case that needs a prediction, such as one patient whose influenza admission risk must be estimated.
2	**Random subset**	Each tree is trained from a subset of information, so different trees can learn different signals.
3	**Trees**	Tree 1, Tree 2, and Tree n each return a result for the same case.
4	**Weighting**	The separate tree results are weighted so stronger, or more useful tree outputs influence the final answer more.
5	**Final result**	The weighted results become one final prediction, such as the predicted influenza hospital admissions.

Core XGBoost Terms: Errors and Starting Point

Residuals

Current prediction error for each row.

Initial guess

The starting prediction before any trees are added.

Loss function

A score that measures how wrong the prediction is.

Core XGBoost Terms: Tree Parts

Root

The first node where the tree starts.

Split

A rule that divides rows into two groups.

Branch

The path from one tree node to another.

Leaf

A final node that stores a prediction output.

Left leaf

The terminal group reached from the left side of a split.

Right leaf

The terminal group reached from the right side of a split.

Core XGBoost Terms: Training Controls

Gain

The improvement produced by a split.

Depth

The number of decision levels in the tree.

Pruning

Removing branches that do not help enough.

Gamma

The minimum gain needed to keep a branch.

Learning rate

Controls how strongly each tree changes the model.

XGBoost Classification: Intuition (1)

▲ *Let's follow the algorithm:*

1. Calculate the residuals based on initial guess of 0.5

2. Make the first split (average of last two observations) and calculate

   $$\text{Similarity score} = (\Sigma \text{ Residuals})^2 / (\Sigma[\text{Previous probability} \times (1 - \text{Previous probability})] + \lambda) \text{ of each leaf}$$

3. Calculate Gain = Left_similarity + Right_similarity – Root_similarity

4. Continue splitting with highest gain split. Default depth is 6 levels

XGBoost Classification: Intuition (2)

5 Prune the trees by calculating Gain - γ (3 by default). If negative, we prune the branch

6 Calculate Output =

$$\Sigma \text{ Residuals} / (\Sigma[\text{Previous probability} \times (1 - \text{Previous probability})] + \lambda)$$

7 Find Predicted values = log odds of default + Learning rate (0.3 by default) × Output

8 Go to step 1: calculate new residuals from the new predicted values

Different Loss function $L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$

XGBoost Regression: Intuition (1)

▲ *Let's follow the algorithm:*

1. Calculate the residuals

2. Make the first split of residuals and calculate

   $$\textit{Similarity score} = (\Sigma\ \textit{Residuals})^2 / (\textit{Number of Residuals} + \lambda)$$

3. Find Gain = Left_similarity + Right_similarity – Root_similarity. Leave the first split in a form which maximizes gain

4. Continue splitting. Default depth is 6 levels

XGBoost Regression: Intuition (2)

5 Prune the trees: if Gain - γ (3 by default) < 0, we prune

6 Calculate Output =

$$\Sigma \text{ Residuals} / (\text{Number of Residuals} + \lambda)$$

7 Find Predicted values = Default (0.5 by default) + Learning rate × Output

8 Go to step 1: calculate new residuals from the new predicted values

Advantages

XGBoost includes several features and characteristics that make it useful in many scenarios:

Scalable for large datasets with millions of records.

Supports parallel processing and GPU acceleration.

Offers customizable parameters and regularization for fine-tuning.

Includes feature importance analysis for better insights.

Available across multiple programming languages and widely used by data scientists.

XGBoost also has certain aspects that require caution or consideration:

Computationally intensive; may not be suitable for resource-limited systems.

Sensitive to noise and outliers; careful preprocessing required.

Can overfit, especially on small datasets or with too many trees.

Limited interpretability compared to simpler models, which can be a concern in fields like healthcare or finance.

Cannot extrapolate beyond the range of values seen in training — as a tree-based model, forecasts are bounded by observed data and cannot capture trends outside that range.

Example Model Overview(s)

ARIMA — Statistical

Models a series from its own past — autocorrelation, trend (via differencing), and seasonality.

GAM — Statistical

Flexible regression using smooth spline terms to capture nonlinear trends and seasonality.

XGBoost — Machine learning

Gradient-boosted decision trees that learn from engineered features and lagged predictors.

Ensemble — Meta-model

Combines several models' forecasts (mean, median, or weighted) — usually the most robust.

Ensembles: Combining Models

Combine forecasts from several models into one prediction that's more robust than any single model on its own.

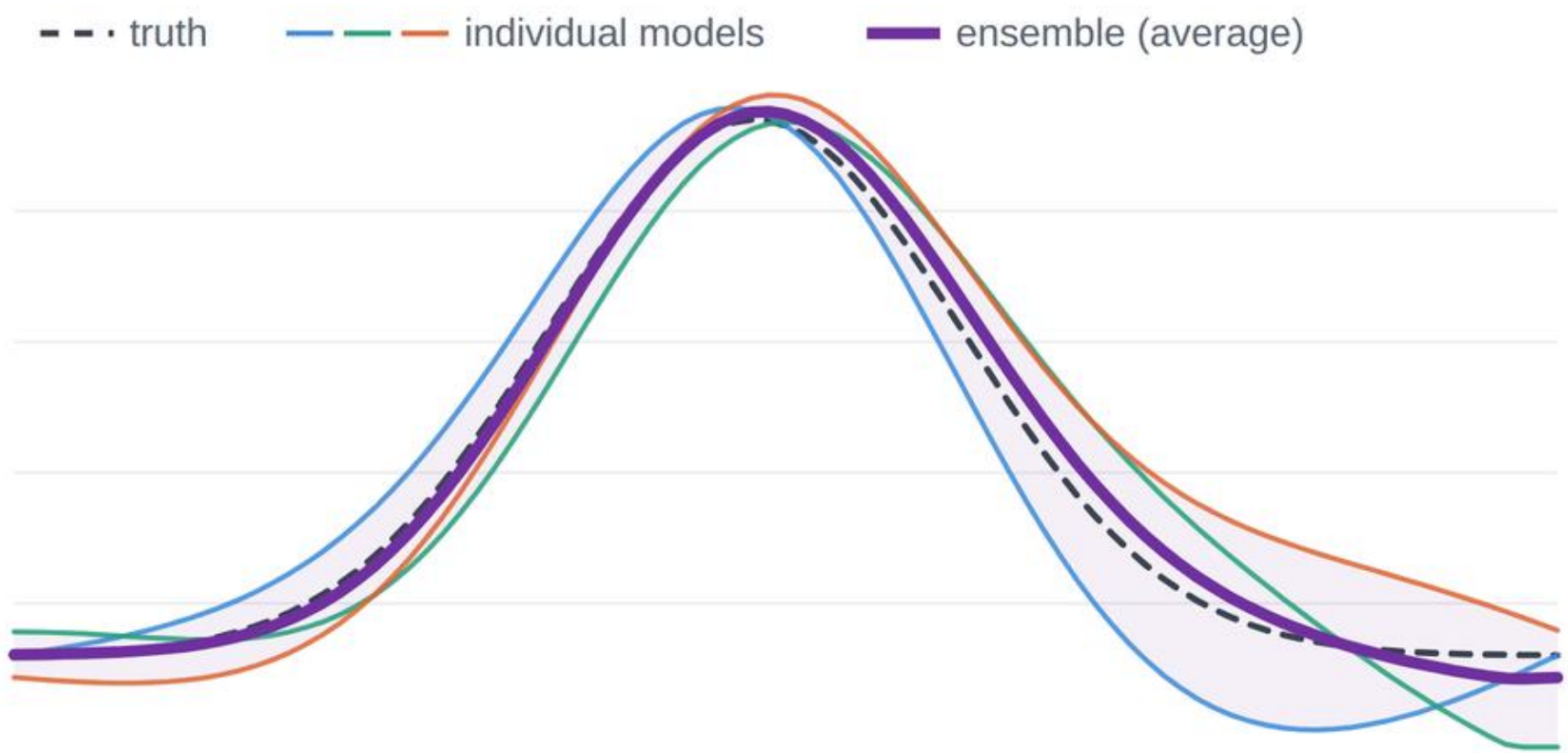
- There are a multitude of ways to combine forecasts, including:
  - **Average** or **median** of the member forecasts.
  - **Weighted combination**: members weighted by past skill.
  - **Stacking**: a model learns the weights.

Why ensembles work well

- Models make different mistakes, so their errors partly cancel.
- No single model is best every season, so the ensemble hedges instead of betting on one.
- The result is steadier across changing conditions, with better-calibrated intervals.

Why an ensemble is more robust

Each model misses in its own way, so their errors partly cancel. The ensemble — the average of the members — stays close to the truth even as the individual models swing.

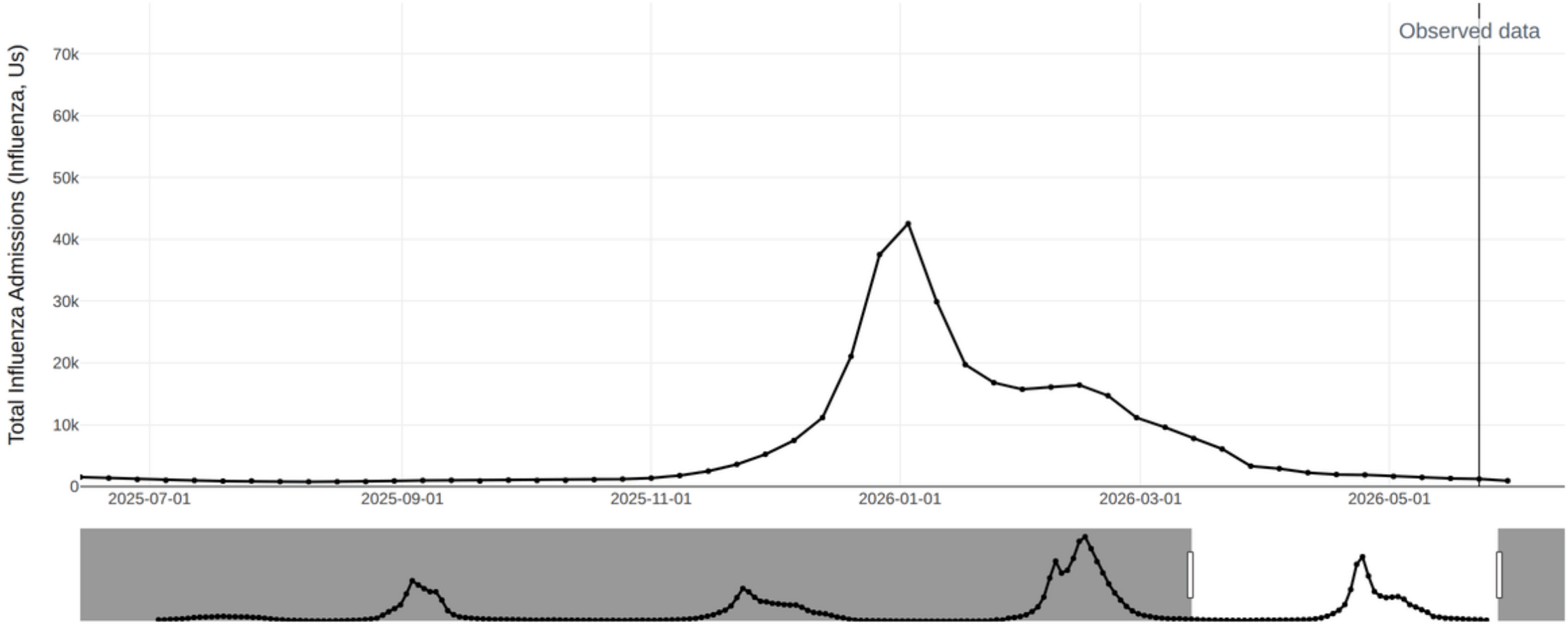


In ID forecasting: the CDC FluSight ensemble works this way — consistently among the most reliable forecasts, though it can still lag sudden changes.

Generated with Claude, for illustration purposes only.

Example: US Influenza Forecasts — FluSight Ensemble (2025-2026 Season)

The FluSight ensemble blends weekly U.S. influenza-hospitalization forecasts from dozens of independent teams into one probabilistic forecast, taken as the median across all submitted models at each quantile.
Each gold line is one week's median forecast with shaded prediction intervals; laid over the observed admissions (black), they show how consistent the ensemble's outlook stayed across the 2025-2026 season.



Strengths, Weaknesses, and When to Use Each Model

Model	Strengths	Weaknesses	Optimal Use Case
ARIMA	Simple, fast, strong for short-term trends and seasonality	Assumes linear patterns; struggles with sudden shifts or extra predictors	Short-term forecasts from a single clean time series
GAM	Flexible curves capture nonlinear trends; stays interpretable	Can overfit; needs careful smoothing choices	Nonlinear trends where you still want to explain the drivers
XGBoost	Handles many predictors and complex interactions; high accuracy	Needs more data; harder to interpret	Rich data with many features and nonlinear effects
Ensembles	Combine models to boost accuracy and stability	More complex to build and run; less transparent	When robust, reliable forecasts matter most

Incorporating Predictors

- **Predictors add external signal** beyond the outcome's own history.
- **Target history** tells us "what has happened."
- **Predictors** tell us "what may be changing."
- **Availability:** predictors must be known at forecast time.
- **For example,** signals that can lead hospitalizations:
  - Influenza RNA in wastewater may precede in-patient hospitalizations by ~2 weeks
  - Influenza testing / positive labs may precede in-patient hospitalizations by 2 days to 2 weeks

ARIMA: Incorporating Predictors

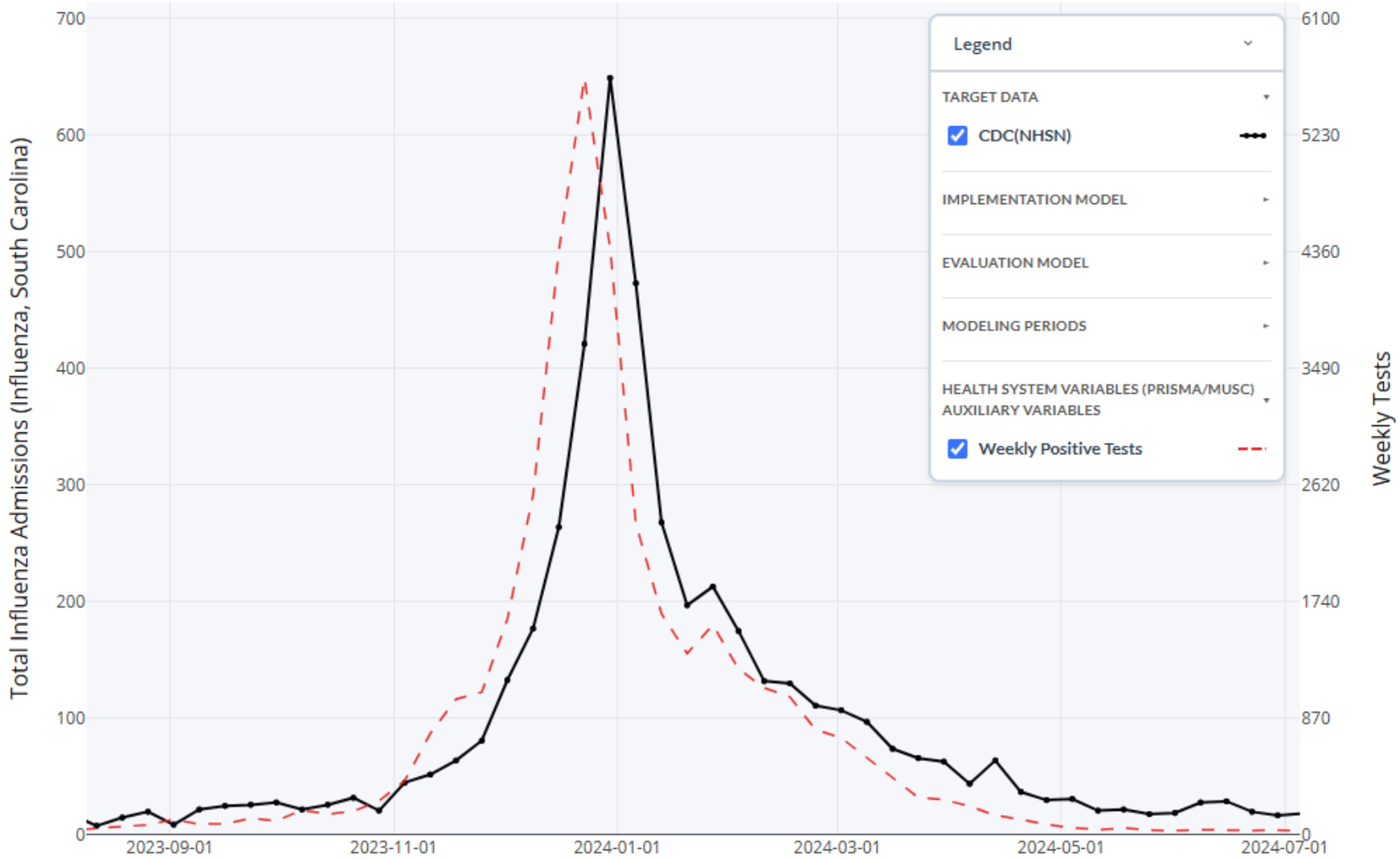
Formula:

$$y'_t = \alpha + \sum \phi_i y'_{t-i} + \sum \theta_j \epsilon_{t-j} + \beta x_{t-\ell} + \epsilon_t$$

y_t	Weekly influenza admissions.
$x_{t-\ell}$	The lagged wastewater signal.
β	The estimated effect of the predictor.
ℓ	The lag chosen based on plausibility/performance.

For example, x_{t-l} may represent wastewater RNA concentration from 2 weeks ago ($l = 2$) or positive labs for influenza based on EHR records from one week ago ($l = 1$)

Positive Tests as a Leading Indicator (South Carolina)



Training and Testing Periods

Training

The past data the model learns from. It studies examples where the outcome is already known and adjusts to capture the patterns.

Like studying practice problems with the answer key.

OUR TRAINING PERIOD

2020-08-08 → 2025-05-16

Rolling Window (grows one week at a time)

Testing

Fresh data the model has not seen. We use it to check how well the model really forecasts, by comparing its predictions to the real outcomes.

Like sitting an exam with new questions.

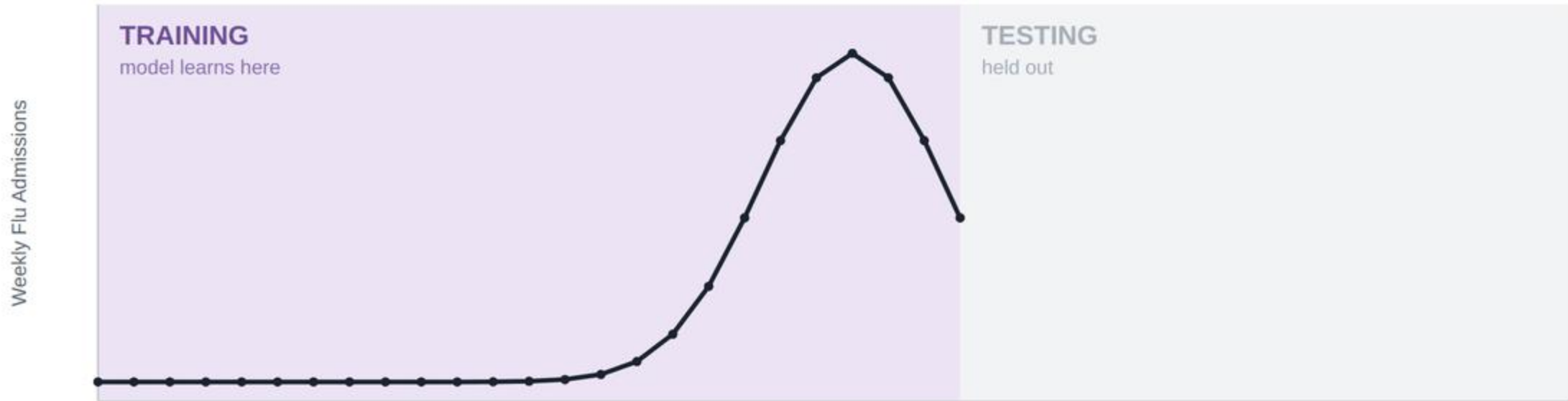
OUR TESTING PERIOD

2025-10-04 → 2026-06-06

Rolling Window; Held out to score the forecasts

Training vs. Testing Periods

The model **learns** from the training period, where it can see the outcomes. On the testing period it **forecasts weeks it never saw**; only afterward do we reveal what actually happened and measure the error.



Training: the model studies these weeks and fits its parameters — it can see the actual admissions.

Overview: Model Evaluation

- Once a model is fit, we measure how well its forecasts perform, judging both point accuracy and how well the model captures uncertainty.
- Three metrics do this, one for the center of the forecast, one for its spread, and one for overall forecast skill:

Mean Absolute Error

POINT ACCURACY

Average absolute gap between the median forecast and what actually happened. Lower is better.

95% PI Coverage

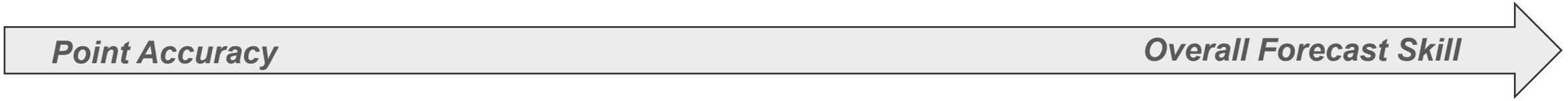
CALIBRATION

How often the truth falls inside the 95% interval. Should land near 95%.

Weighted Interval Score

OVERALL SKILL

A single score across all quantiles that rewards forecasts that are both sharp and well-calibrated.



In short: MAE checks accuracy, coverage checks the calibration, and WIS combines sharpness (interval width) and calibration into one score.

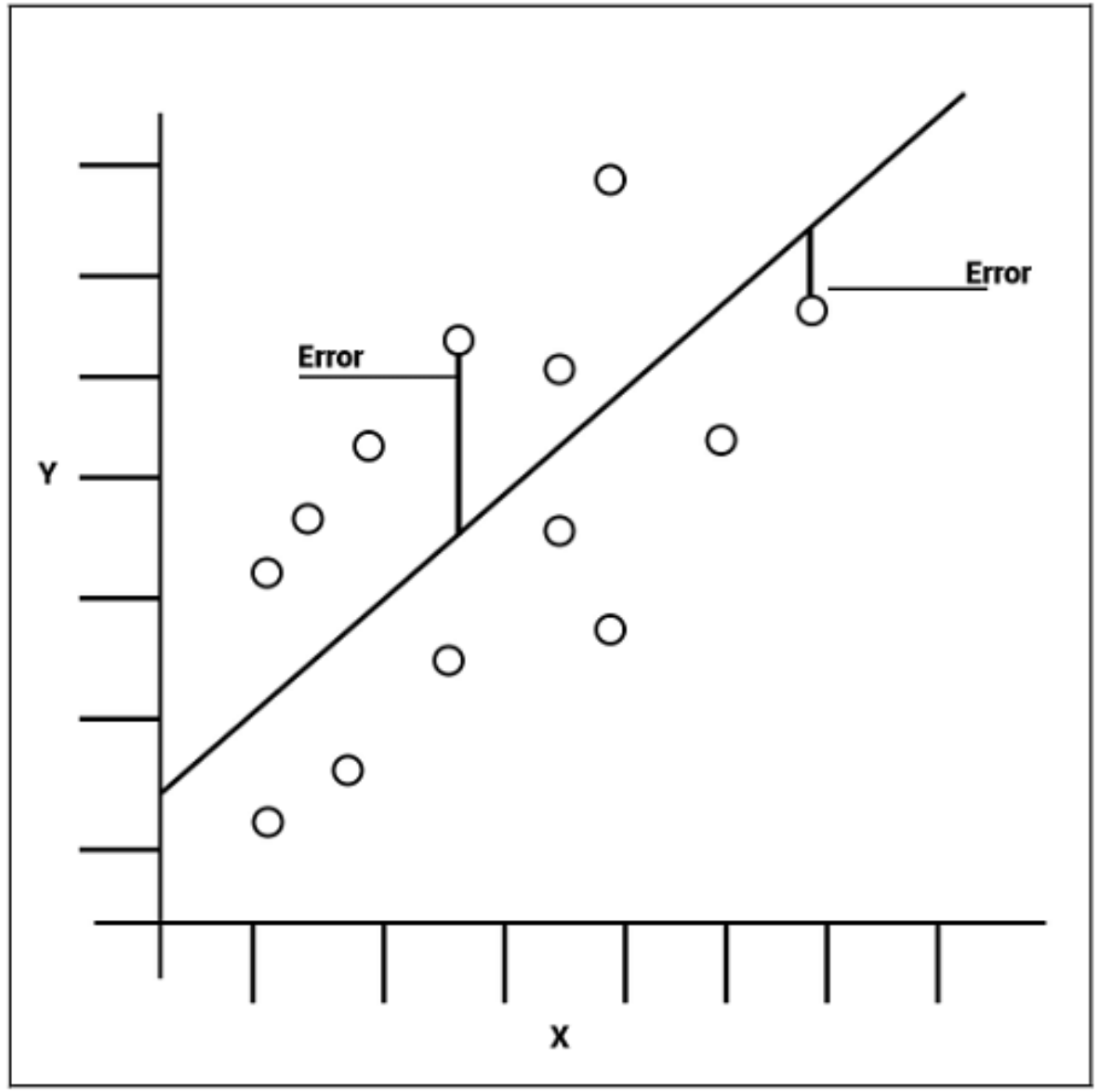
Mean Absolute Error (MAE)

- Measure of how **accurate** point (single value per time point) forecasts are (i.e., how far the model's predictions fall from the observed case counts).

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

N	Number of observations
y_i	Observed value
$\hat{y}_i$	Predicted value (e.g., median forecast value)

Ideal Value: Lower is better!


Error
Error
Y
X

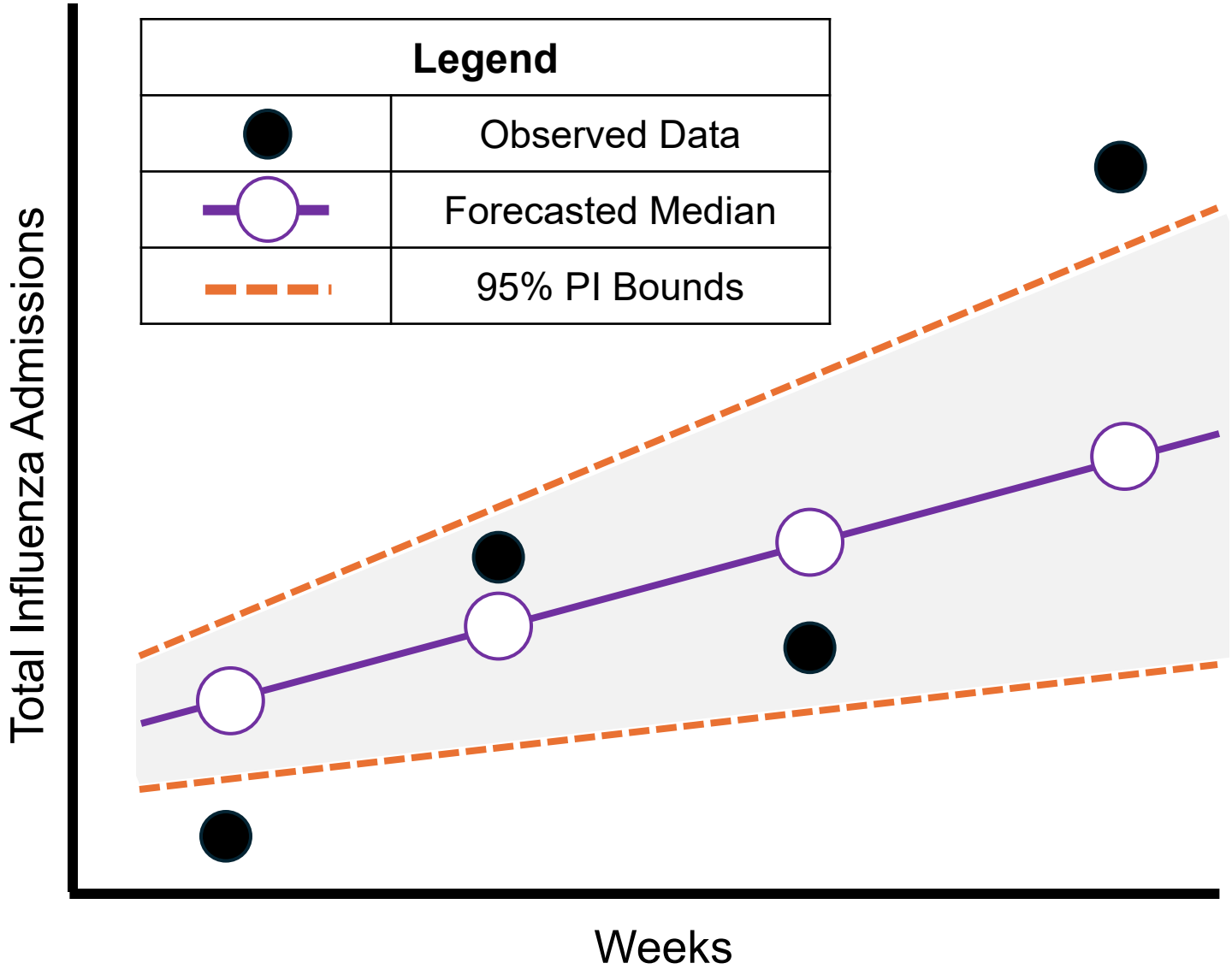
95% Prediction Interval (PI) Coverage

- Measures how often the observed value is captured within the bounds of the 95% prediction interval (i.e., measure of how well the **model captures uncertainty**).

$$95\%\ PI\ Coverage = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(l_i \leq y_i \leq u_i)$$

N	Number of observations
$\mathbf{1}(\cdot)$	Indicator (1 if the observation falls in the interval, 0 otherwise)
y_i	Observed value
l_i, u_i	Lower/Upper bounds of the 95% PI

Ideal Value: ~95%; Below 95% = intervals too narrow (overconfident); above 95% = too wide (underconfident).



Calculating the 95% PI Coverage

1. We are working with four dates of data (N = 4).
2. For 2 of those 4 dates, the observed data falls within the bounds.

$$\frac{\#\ Observed\ data\ dates\ within\ bounds}{Total\ Number\ of\ Dates\ (N)}$$

$$= \frac{2}{4} = \mathbf{0.5\ or\ 50\%\ PI\ Coverage}$$

Interval Score (IS)

- Measures how well a **single** prediction interval performs, rewarding intervals that are narrow but penalizing ones that the observed value falls outside, in the units of the observed data.

$$IS_\alpha(F, y) = \underbrace{(u - l)}_{\text{Dispersion}} + \overbrace{\frac{2}{\alpha} * (l - y) * \mathbf{1}(y \leq l)}^{\text{Overprediction}} + \underbrace{\frac{2}{\alpha}(y - u) * \mathbf{1}(y \geq u)}_{\text{Underprediction}}$$

l, u	Lower/Upper bounds of prediction interval
y	Observed value
$\mathbf{1}(\cdot)$	Indicator becomes 1 if condition within is TRUE
α	Significance level (e.g., 0.05 for a 95% interval)

Ideal Value: Lower is better!

Weighted Interval Score (WIS)

- Measures overall forecast quality, focusing on how close the whole predicted distribution is to the observed value, in the units of the observed data. It rewards forecasts that are both **accurate** and **appropriately confident**.

$$WIS = \frac{1}{K + 0.5} * \left(0.5 * |y - m| + \sum_{k=1}^{K} w_k * IS_{\alpha_k}(F, y) \right)$$

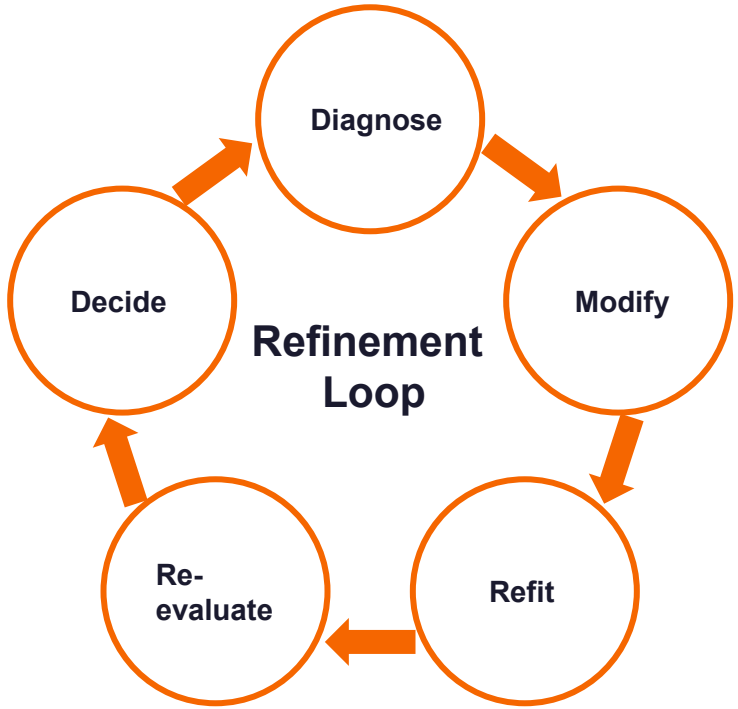
K	Number of Prediction Intervals sets
y	Observed value
m	Forecasted Median
w_k	Weight Assigned to each interval; $\mathrm{w_k} = \frac{\alpha_k}{2}$
$IS_{\alpha_k}(F, y)$	Interval score

Ideal Value: Lower is better!

Model Refinement: An Iterative Loop

In infectious disease forecasting, refinement means improving the forecasting system based on what we learn from evaluation.

1. **Diagnose** — Where does performance break down? By horizon, season phase, geography, outcome, or uncertainty.
2. **Modify** — Make a targeted change to the data, predictors, model structure, or uncertainty estimates.
3. **Refit** — Train the revised model using the same forecasting setup.
4. **Re-evaluate** — Compare forecasts against held-out observations.
5. **Decide** — Keep the refinement only if it improves out-of-sample forecast performance (i.e., in testing period).
    - **Example:** if flu hospitalization forecasts become less reliable at longer horizons, we may test whether adding wastewater improves 2– or 3-week-ahead forecasts.



Note: refinement should improve future forecasts, not just make the model fit past data better.

Model Refinement: What Can We Refine?

- **Data & observation process** — reporting delays, missingness, revisions, outliers, transformations, nowcasting.
- **Predictors & features** — lagged outcomes, tests/labs, wastewater, digital traces, mobility, vaccination, weather, calendar terms, rolling averages.
    - Predictor transformations and interactions can also be explored (e.g., slope of past 3 lagged outcomes, log(wastewater), wastewater*septic tank use interaction)
- **Model structure** — seasonality, lag structure, nonlinear terms, model class, hyperparameters.
- **Training strategy** — rolling vs. expanding windows, recency weighting, geographic pooling, season-specific training.
- **Uncertainty estimates** — prediction intervals, quantile forecasts, residual assumptions, calibration, ensemble weighting.
- **Example (flu forecasting):** add lagged wastewater trends, or move from ARIMA to XGBoost.
- **Key idea:** each refinement should target a weakness found during evaluation.

Model Refinement: General Strategy

- **Start with a baseline** — a simple, reproducible model as the benchmark.
- **Identify the weakness** — use evaluation metrics and plots to find where forecasts fail.
- **Change one major component** — data, predictors, model form, training setup, or uncertainty method.
- **Compare fairly** — same forecast dates, targets, horizons, and testing period.
- **Assess tradeoffs** — a change may improve MAE but worsen PI coverage or help one horizon while hurting another.
    - **Example:** compare ARIMA, ARIMA + wastewater, and XGBoost on the same rolling test period using WIS, MAE, and 95% PI coverage.
- **Bottom line:** the best refinement improves forecast reliability across multiple settings under realistic conditions (e.g., multiple training/testing windows across seasons).

Working Example
CDC NHSN Hospital Respiratory Data

Overview

- Every week during influenza season, public health and healthcare decision-makers need to anticipate what's coming:
  - How many people will be hospitalized next week, and in the weeks ahead?
  - When will the peak hit and how high will it be?
- In this working example, you'll walk through a national-level influenza hospitalization forecasting system using real CDC data to help answer these questions.
  - The hands-on lab will focus on a **human-AI teaming approach** to achieve the same (or similar results).

CDC (NHSN) Hospital Respiratory Data

The weekly surveillance dataset behind our national flu forecasting pipeline.

What it is

Weekly hospital respiratory data from CDC's National Healthcare Safety Network (NHSN), aggregated to the national (USA) and state or territory level, going back to August 2020.

What it captures

New admissions, hospital occupancy, and bed capacity for the three main respiratory pathogens: COVID-19, **influenza**, and RSV.

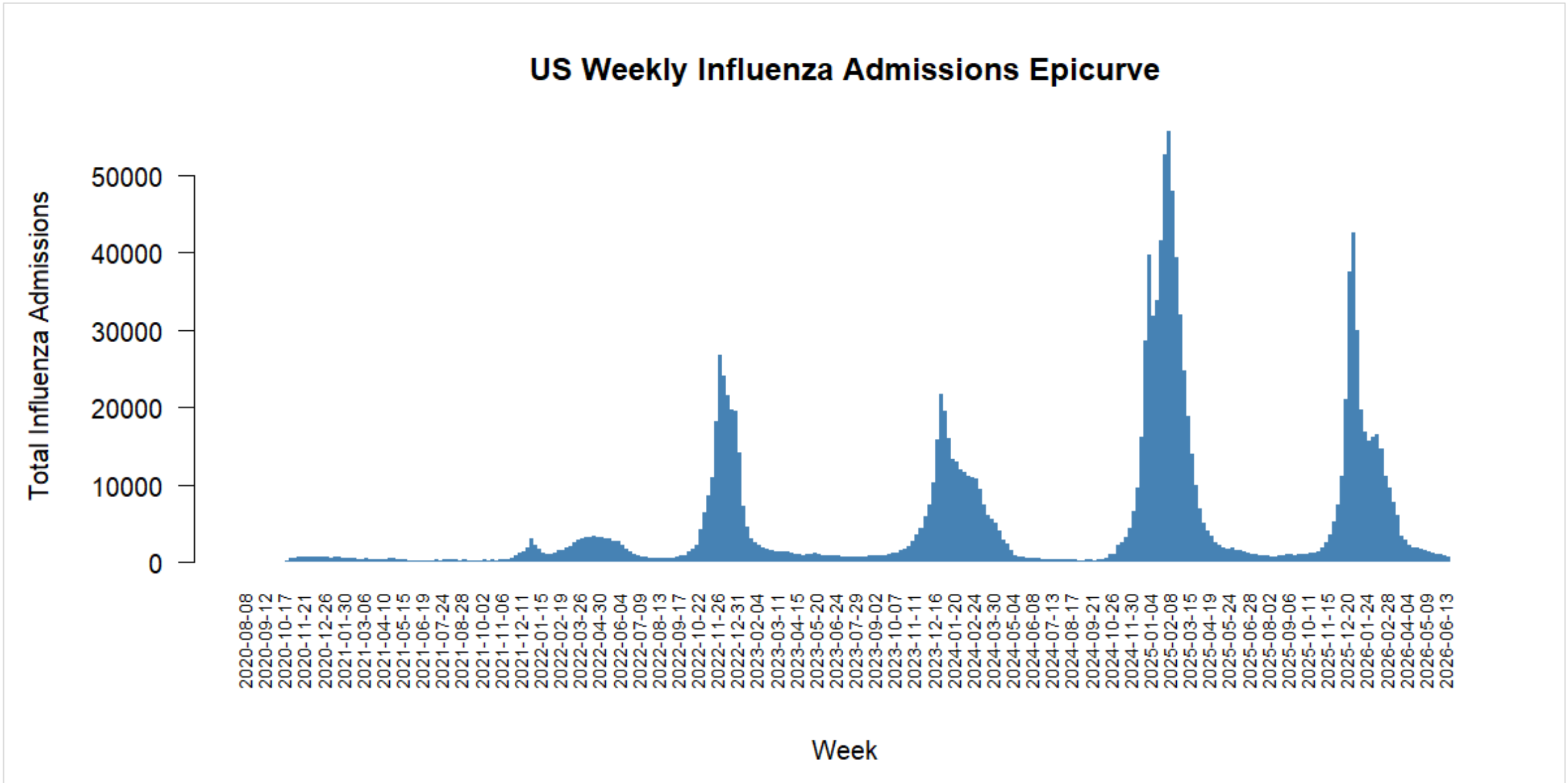
Reporting cadence

Published weekly by MMWR week as a preliminary release that is later finalized, so the most recent weeks can still revise before they settle.

How we use it

We keep the `Total Influenza Admissions` metric for the `USA` jurisdiction and clean it into a tidy weekly series. That cleaned series is the starting point for the entire forecasting pipeline.

Source: CDC National Healthcare Safety Network (NHSN) · data.cdc.gov

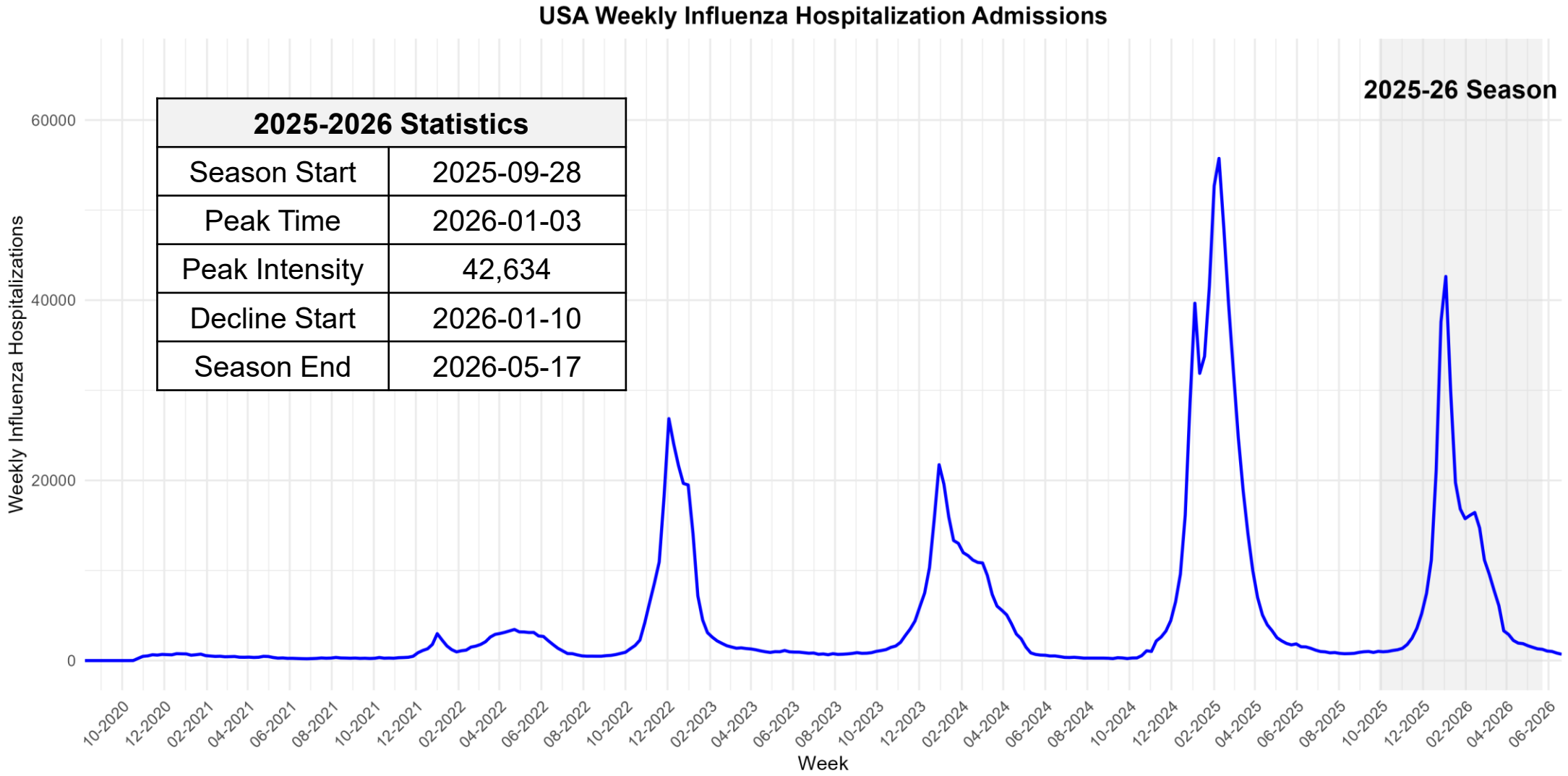

US Weekly Influenza Admissions Epicurve
Total Influenza Admissions
50000
40000
30000
20000
10000
0
2020-08-08
2020-09-12
2020-10-17
2020-11-21
2020-12-26
2021-01-30
2021-03-06
2021-04-10
2021-05-15
2021-06-19
2021-07-24
2021-08-28
2021-10-02
2021-11-06
2021-12-11
2022-01-15
2022-02-19
2022-03-26
2022-04-30
2022-06-04
2022-07-09
2022-08-13
2022-09-17
2022-10-22
2022-11-26
2022-12-31
2023-02-04
2023-03-11
2023-04-15
2023-05-20
2023-06-24
2023-07-29
2023-09-02
2023-10-07
2023-11-11
2023-12-16
2024-01-20
2024-02-24
2024-03-30
2024-05-04
2024-06-08
2024-07-13
2024-08-17
2024-09-21
2024-10-26
2024-11-30
2025-01-04
2025-02-08
2025-03-15
2025-04-19
2025-05-24
2025-06-28
2025-08-02
2025-09-06
2025-10-11
2025-11-15
2025-12-20
2026-01-24
2026-02-28
2026-04-04
2026-05-09
2026-06-13
Week

Exploring the Data

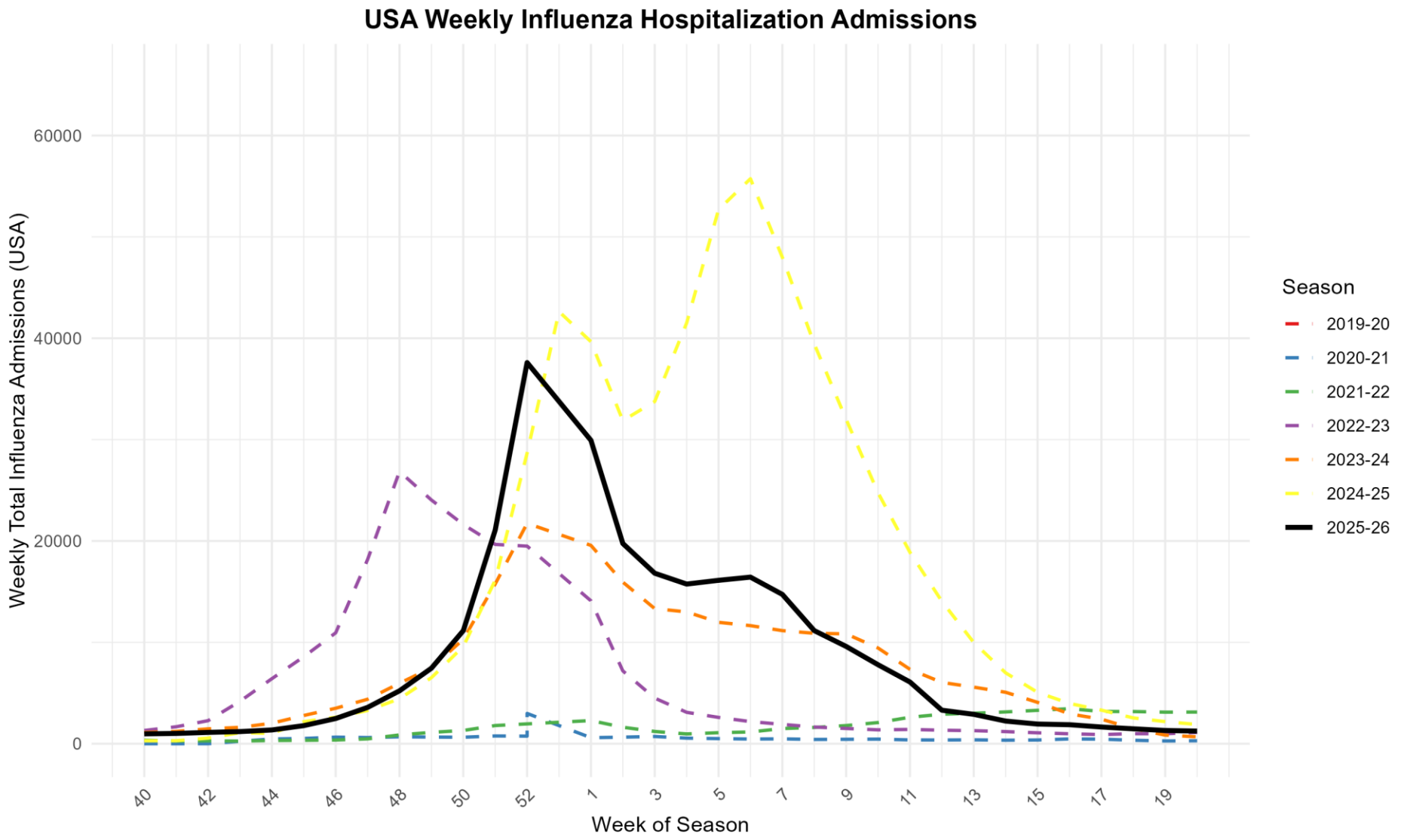
- Before fitting a model and forecasting, we take a close look at the data to understand current and past trends, along with key epidemiological features such as peak timing and intensity.
- We are retrospectively forecasting the 2025-2026 season, which raises a few questions:
  - How do we define a season?
  - How does this season compare to past seasons?
  - When did key turning points occur (peak timing), and how intense were they (peak intensity)?

Why this matters: Exploring the data, through both visuals and summary values, is how we build a full picture of what is going on before we commit to a model.

National Trend, Current Season Highlighted



Current Season vs Past Seasons

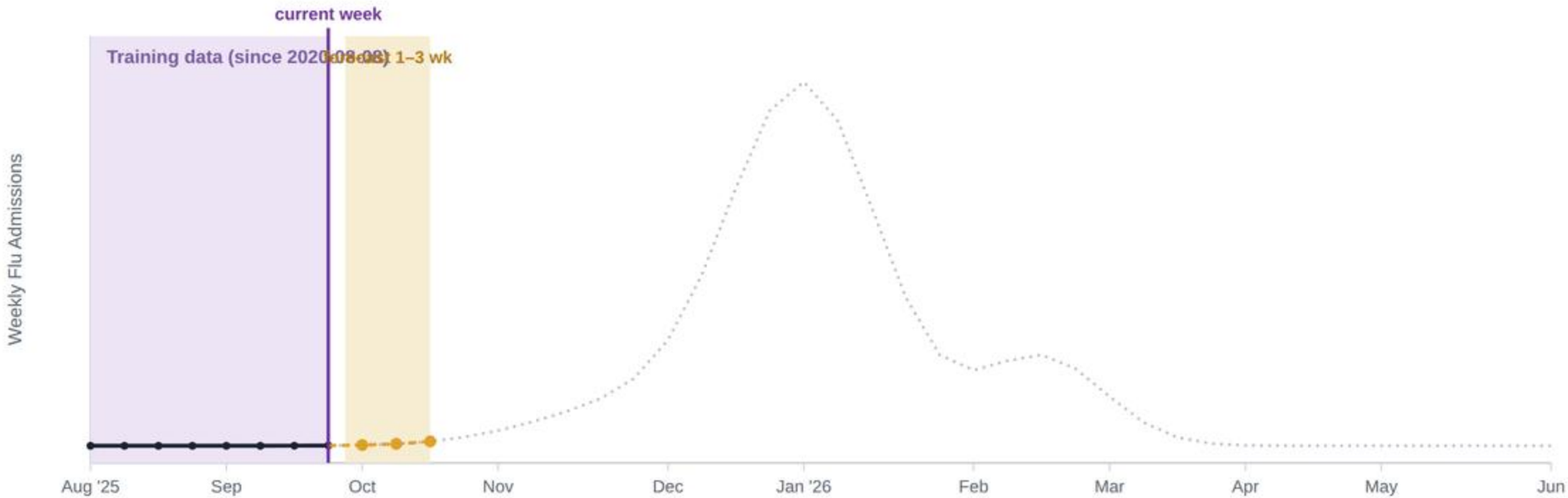


First Pass: Model Building and Forecasting

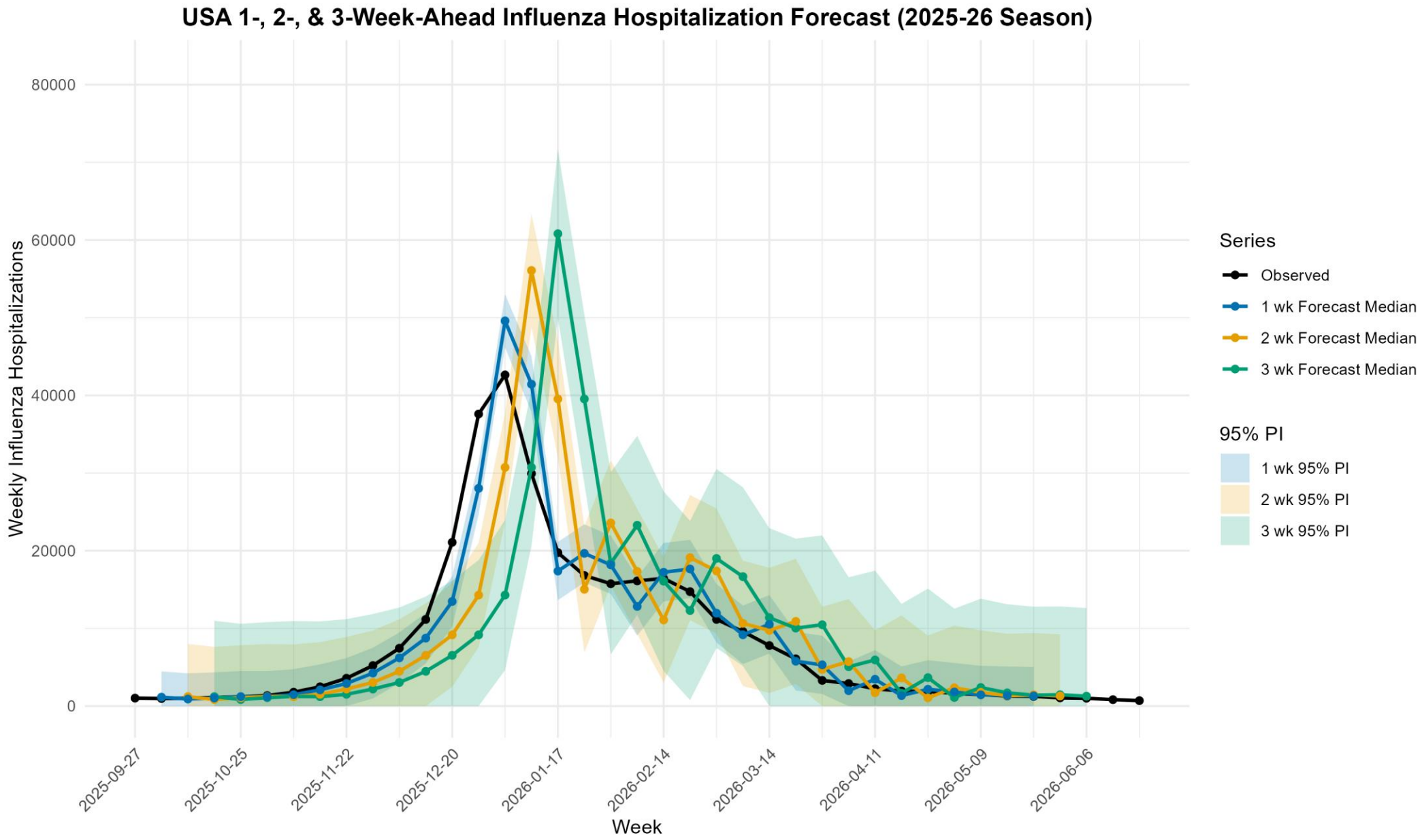
Model	ARIMA
“Current Week”	Rolling Window (2025-09-27 through 2026-05-16)
Target Data	CDC NHSN Data
Auxiliary Data	None
Outcome	National Weekly Total Flu Admissions
Training Period (Rolling Window)	2020-08-08 through 2026-05-16
Testing Period	2025-10-04 through 2026-06-06
Horizons	1, 2, & 3 Weeks

Rolling Window: Training Grows Each Week

Each week, the newly observed week is added to the training data, the model is refit, and it forecasts 1–3 weeks ahead. The forecast origin rolls forward one week at a time across the season.



Current week: 2025-09-27 · **Training window:** 2020-08-08 → 2025-09-27 (one week added each step)

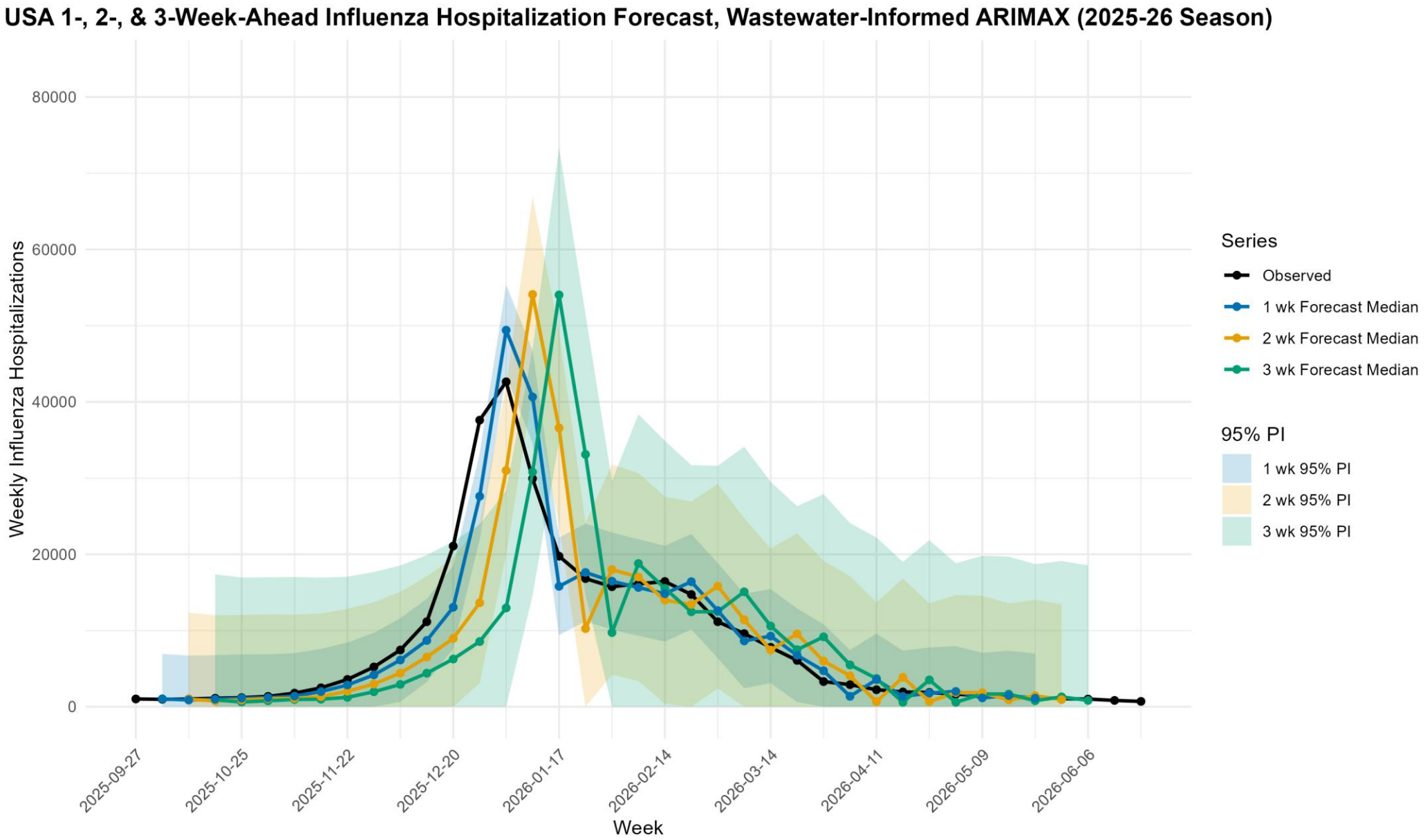

USA 1-, 2-, & 3-Week-Ahead Influenza Hospitalization Forecast (2025-26 Season)
Weekly Influenza Hospitalizations
80000
60000
40000
20000
0
2025-09-27
2025-10-25
2025-11-22
2025-12-20
2026-01-17
2026-02-14
2026-03-14
2026-04-11
2026-05-09
2026-06-06
Week
Series
Observed
1 wk Forecast Median
2 wk Forecast Median
3 wk Forecast Median
95% PI
1 wk 95% PI
2 wk 95% PI
3 wk 95% PI

Forecast (Testing Period) Evaluation

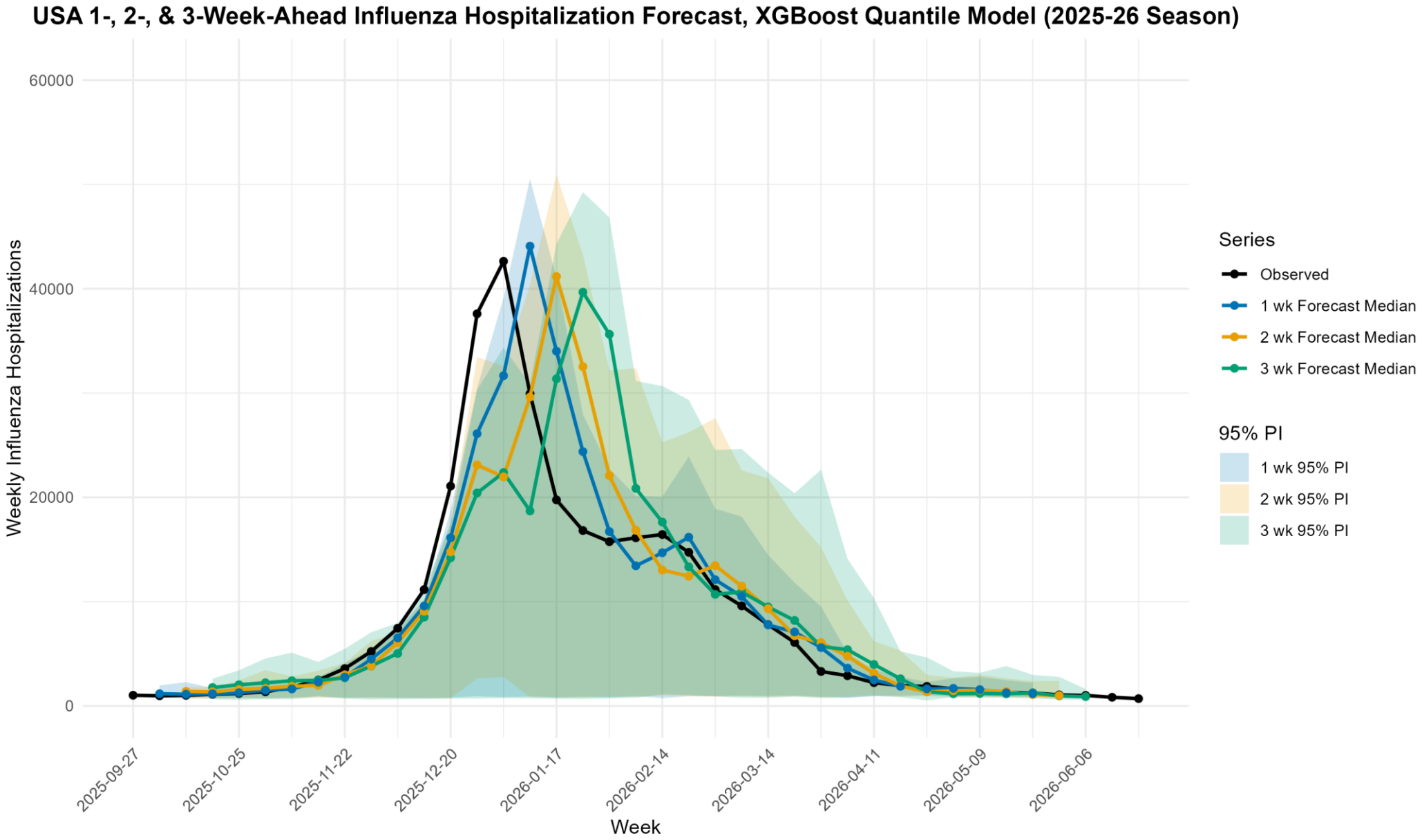
- Evaluation tells us how much we can trust a forecast. By comparing predictions against what actually happened, we can see which models are reliable and where they tend to go wrong.
- The table below shows each metric averaged across all forecast periods (the separate windows of time we made forecasts for), so every number summarizes performance over many forecasts, not just one.
- Testing period range: 2025-10-04 → 2026-06-06

Horizon	WIS (Range)	MAE (Range)	95% PI Coverage
1	1529.8 (312.5 – 10129.1)	1954.2 (16 – 11505)	0.88
2	3493.9 (532.3 – 23364.3)	4407.7 (34 – 26148)	0.85
3	4862.8 (705.1 – 36917.7)	6102.2 (99 – 41058)	0.85

Refinement: Added Indicator



Refinement: Trying XGBoost



Evaluation Example: 3-week Forecast

- Suppose our interest lies in the 3-week ahead forecasts
- We measure each change against the current model using common evaluation metrics, then decide whether to keep it or revert.

Change	Model	Indicator	WIS	MAE	95% PI Coverage	Keep?
Baseline	ARIMA	N/A	4862.8	6102.2	85%	—
Leading Indicator	ARIMA	National WVAL	4271.1	5407.0	91%	[Y / N]
Model Swap	XGBoost	N/A	2634.8	4179	91%	[Y / N]

These evaluation metrics were averaged across all 3-week forecasts over the testing period. Are these sufficient? Are there other metrics we could have used?